

PROJECT REPORT

on

**INTELLIGENT CONVERSATIONAL AI CHATBOT
FOR AUTOMATED CUSTOMER SUPPORT —
BILLING DISPUTES AND PERSONALIZED OFFER
RECOMMENDATIONS**

Submitted in partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications (MCA)

By

Student Name: _____

Enrollment No: _____

Under the guidance of

Guide / Mentor Name: _____

Department of Computer Applications

Academic Year 2025–2026

Bonafide Certificate

This is to certify that the project report entitled “**Intelligent Conversational AI Chatbot for Automated Customer Support — Billing Disputes and Personalized Offer Recommendations**” is a bonafide record of the project work carried out by _____ (Enrollment No. _____) in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications, under my supervision and guidance during the academic year 2025–2026. The work reported herein is original and has not been submitted, in whole or in part, for the award of any other degree or diploma.

Guide / Mentor

Head of the Department

Name: _____

Name: _____

Copy of the certificate received from Qollabb to be pasted here.

Declaration

I, _____, hereby solemnly declare that the project report titled “**Intelligent Conversational AI Chatbot for Automated Customer Support — Billing Disputes and Personalized Offer Recommendations**” submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2025–2026 under the supervision of _____ (Guide / Mentor Name).
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated. (All customer data used by the system is synthetic and created for demonstration purposes.)
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: _____

Date: _____

Student Signature: _____

Student Name: _____

Enrollment No: _____

Acknowledgement

I take this opportunity to express my sincere gratitude to my project guide and mentor for their invaluable guidance, encouragement and continuous support throughout the course of this project. Their insights into machine learning, conversational systems and software engineering were instrumental in shaping this work and in helping me navigate the many design decisions it involved. I extend my heartfelt thanks to the Head of the Department and to the faculty of the Department of Computer Applications for providing the academic environment, infrastructure and timely feedback that made this project possible. I am also grateful to my peers for their constructive criticism during design reviews and demonstration rehearsals, which materially improved the quality of the final system. Finally, I thank my family for their unwavering support, patience and motivation during the development of this project. This work would not have been possible without the collective encouragement of all these individuals, and I remain deeply indebted to them.

Abstract

This project, titled “**Intelligent Conversational AI Chatbot for Automated Customer Support,**” designs and implements a text-based conversational agent that resolves two of the highest-volume telecom support intents: **billing disputes** and **personalized offer recommendations**. Its **objective** is to deliver automated, accurate, real-time support that understands natural-language queries, acts on the customer's account, escalates difficult cases, and improves from feedback. The system uses a set of **technologies** centred on Python: a Streamlit chat interface; deep-learning Natural Language Understanding built from Multi-Layer-Perceptron neural networks over TF-IDF features for intent and sentiment classification, with rule-based entity extraction; a grounded Natural Language Generation module backed by an optional local Large Language Model (Ollama); a deterministic, rule-based recommendation engine; and an epsilon-greedy reinforcement-learning bandit. Architecturally it follows a **hub-and-spoke multi-agent** design in which a coordinator routes each turn between a billing-and-offers agent and a retention specialist. The **development methodology** was iterative and incremental, executed across six milestones from data collection to deployment and monitoring, with Git version control. The **key results** are a working, single-command application achieving approximately 83% sentiment accuracy, a 100% resolution rate on the designed test scenarios, low per-turn latency, and grounded, hallucination-free replies, thereby satisfying all five project objectives while remaining fully local, reproducible and privacy-safe.

Table of Contents

Chapter	Title
—	Bonafide Certificate · Declaration · Acknowledgement · Abstract · List of Figures · List of Tables · List of Abbreviations
1	Introduction
2	Literature Review / System Study
3	System Analysis
4	System Design
5	System Implementation
6	Testing
7	Results & Discussion
8	Conclusion & Future Enhancements
9	References
10	Appendices

List of Figures

Figure	Caption
3.1	System Architecture (high level)
3.2	Data-Flow Diagram — Level 0 (Context)
3.3	Data-Flow Diagram — Level 1
3.4	Use-Case Diagram
4.1	Subsystem Decomposition
4.2	Per-Turn Processing Pipeline
4.3	Hub-and-Spoke Structure
4.4	Class Diagram
4.5	Entity-Relationship Diagram
4.6	Activity Diagram — Billing Dispute
4.7	Activity Diagram — Offer Recommendation with RL
4.8–4.12	Sequence Diagrams — UC-00 to UC-04
5.1–5.2	Application Screenshots (welcome, bill explanation)
7.1–7.3	Result Screenshots (offer, handoff, resolution)

List of Tables

Table	Caption
2.1	Comparison of Design Approaches
3.1	Functional Requirements
3.2	Non-Functional Requirements
4.1	Subsystem Responsibilities
4.2	Data Schema — Table Structures
5.1	Hardware and Software Requirements
5.2	Milestone Outcomes
6.1	Test Cases
7.1	NLU Metrics
7.2	Comparison with the Existing System

List of Abbreviations

AI — Artificial Intelligence; API — Application Programming Interface; ARPU — Average Revenue Per User; CRM — Customer Relationship Management; CSAT — Customer Satisfaction; DFD — Data-Flow Diagram; DL — Deep Learning; ER — Entity-Relationship; KPI — Key Performance Indicator; LLM — Large Language Model; ML — Machine Learning; MLP — Multi-Layer Perceptron; NLG — Natural Language Generation; NLP — Natural Language Processing; NLU — Natural Language Understanding; PoC — Proof of Concept; RL — Reinforcement Learning; SDLC — Software Development Life Cycle; SLA — Service-Level Agreement; TF-IDF — Term Frequency–Inverse Document Frequency; UAT — User Acceptance Testing; UC — Use Case; UI — User Interface; UML — Unified Modeling Language.

Chapter 1 — Introduction

1.1 Background of the Project

Customer-support contact centres in the telecommunications industry handle enormous volumes of repetitive interactions every day. When the traffic reaching these centres is analysed, two categories of request dominate: customers who are confused about, or wish to contest, a charge that has appeared on their monthly bill, and customers who want to know whether a better plan or a discount is available to them. A subscriber who returns from a trip abroad to find a bill inflated by an international roaming charge, or one who exceeds a monthly data cap and is levied an overage fee, will very frequently contact support in a state of confusion or frustration. In parallel, every operator runs continuous retention and up-sell programmes — loyalty discounts, data add-ons, family-plan consolidations, contract renewals — and surfacing the right offer to the right customer at the right moment is a recognised lever for reducing customer churn and increasing Average Revenue Per User. Both of these workloads are rule-driven and grounded in the customer's account data, which makes them strong candidates for automation, provided the automating system can robustly understand messy, colloquial natural-language input and act without ever fabricating account information.

This project builds precisely such a system: an intelligent, text-based conversational Artificial Intelligence chatbot that automates billing-dispute resolution and personalized offer recommendation for a telecom operator. The system is engineered to run entirely on a laptop against synthetic data, making it fully reproducible and free of the privacy concerns associated with real subscriber records, and it demonstrates how modern machine learning, deep learning and language generation can be combined into a coherent, reliable and observable support agent.

1.1a Real-World Relevance and Economics

A brief consideration of the economics clarifies why the two chosen intents are worth automating well. In telecom, the cost of acquiring a new subscriber is substantial and typically exceeds the cost of retaining an existing one, so any interaction that reduces the risk of a customer leaving carries outsized value. Billing confusion is a recognized driver of dissatisfaction and, ultimately, of churn: a customer who receives an unexpectedly high bill and then endures a long, unhelpful support experience is materially more likely to defect to a competitor. Conversely, a support interaction that resolves the confusion quickly, treats the customer fairly, and — where appropriate — proactively offers a well-matched discount can convert a moment of frustration into a moment of loyalty. The offer side of the system is the natural complement: surfacing a relevant retention offer at the point of contact, rather than through an untargeted mass campaign, both reduces churn and lifts revenue per user. Automating these interactions with an agent that is instantly available, consistent and personalized therefore attacks operating cost and churn simultaneously, which is the real-world commercial rationale underpinning the technical work in this report. The same architecture, moreover, generalizes readily to adjacent domains — utility billing, banking-fee disputes, subscription management — wherever a customer must be understood, served against account data, and retained.

1.2 Problem Statement

The problem addressed by this project is to *design and implement an intelligent conversational chatbot that, given a customer's free-text query, can accurately determine the customer's intent and emotional state, act on the customer's real account to explain or resolve a billing dispute or to recommend and apply a suitable offer, escalate difficult negotiations to a specialist agent, and do all of this in real time while never fabricating account facts and while continuously improving from customer feedback*. This decomposes into several concrete technical challenges: robust intent classification from short, colloquial utterances; sentiment tracking to drive empathetic and escalation behaviour; grounded natural-language generation that stays faithful to account data; a decision engine that encodes billing and offer policy deterministically; a mechanism for transferring control between multiple cooperating agents; a learning loop that adapts recommendations to observed acceptance; and a usable interface that first validates the customer's identity.

1.3 Objectives of the System

The project is guided by five objectives, which also structure its evaluation:

1. To design and develop an intelligent conversational AI chatbot capable of delivering automated, accurate and real-time customer support using advanced Natural Language Processing techniques.
2. To enhance chatbot response quality and efficiency through machine-learning optimization aimed at improving customer satisfaction and reducing average response time.
3. To implement deep-learning-based architectures for Natural Language Understanding and response generation, enabling context-aware and human-like interactions.
4. To integrate reinforcement-learning strategies for continuous chatbot improvement based on user interactions and feedback loops.
5. To evaluate chatbot performance using key metrics such as accuracy, intent-classification precision, response relevance, operational efficiency and user-satisfaction scores.

1.3a How the Objectives Are Measured

Each objective is paired with a concrete, observable measure so that its achievement can be judged rather than merely asserted. The first objective, automated real-time support, is evidenced by the working end-to-end system that resolves billing and offer queries within a single conversation and by the recorded per-turn latency. The second, improved quality and efficiency, is evidenced by the resolution rate, the low latency, the small number of turns to resolution, and the reinforcement-learning-driven improvement in offer relevance. The third, deep-learning understanding and generation, is evidenced by the two Multi-Layer-Perceptron neural classifiers and by the grounded natural-language generation, whose behaviour is demonstrated in the results. The fourth, reinforcement learning from feedback, is evidenced by the persisted bandit policy and the visible evolution of its learned acceptance values. The fifth, evaluation with key metrics, is evidenced by the accuracy, precision, recall and F1 figures for the classifiers and by the conversation-level KPIs. Defining these measures at the outset kept the development honest, since every feature could be related back to the evidence it would eventually need to produce.

1.4 Scope of the Project

The scope is deliberately bounded to keep the project focused and demonstrable. **In scope:** text-based conversation; two-step identity validation; billing-dispute resolution for roaming and overage charges; explanation of bill composition; provisional credits, goodwill waivers and dispute-ticket creation; personalized offer recommendation and application; a hub-and-spoke handoff to a retention specialist that follows an escalating negotiation ladder; a reinforcement-learning feedback loop for offers; live signal and Key-Performance-Indicator monitoring; and a Streamlit web interface. **Out of scope** for this phase: voice/telephony channels; integration with live production billing/CRM back-ends (synthetic adapters are used instead); transfer to a human agent (both agents are AI); a formal supervisor email-approval workflow (approval is simulated in the interface); and multilingual support. These exclusions are revisited as future enhancements in Chapter 8.

1.5 Existing System Overview

In the conventional telecom support model, a customer with a billing query or an interest in offers contacts a human agent by telephone or live chat. The agent authenticates the customer, looks up the account in a Customer Relationship Management and billing system, interprets the problem, consults Standard Operating Procedures to decide on a resolution, and records the outcome; difficult cases are escalated to a supervisor. Some operators augment this with simple menu-driven or keyword-matching bots. These existing approaches suffer from high operating cost, long queueing times during peak load, inconsistent application of policy across agents, limited genuine round-the-clock availability, a reactive rather than proactive stance toward offers, and poor structured observability of the emotional and intent trajectory of a

conversation. Keyword bots in particular are brittle, failing whenever the customer phrases a request unexpectedly, and they possess no mechanism to sense sentiment or to learn.

1.6 Proposed System Overview

The proposed system replaces the rigid keyword bot and offloads the human agent for the two target intents. After validating the customer's identity, it accepts free-text queries and, for each turn, understands the intent and sentiment, extracts entities, decides on an action against the customer's account, and replies in grounded natural language. It resolves billing disputes by explaining charges and offering evidence-backed remedies, and it recommends personalized offers scored against the customer's profile and refined by a learning loop. When a negotiation becomes difficult, a coordinator hands the conversation to a specialist retention agent. Throughout, the system exposes live signals and KPIs. Its advantages over the existing model are instant availability, consistent policy application, proactive and adaptive personalization, structured observability and negligible marginal cost per conversation.

1.7 Technologies Used (Brief Introduction)

The system is implemented entirely in **Python**. The user interface is built with **Streamlit**, which provides native chat components. Natural Language Understanding uses **scikit-learn** to train Multi-Layer-Perceptron neural networks over **TF-IDF** features for intent and sentiment classification, with regular-expression entity extraction. Response generation combines deterministic templates with an optional local Large Language Model served by **Ollama** (the **llama3.2** model). Data is stored as human-readable **JSON** files, model artefacts are persisted with **joblib**, configuration is read from a **.env** file, and **Git** is used for version control. Each of these choices is justified in detail in Chapter 5. This chapter has introduced the problem, the objectives and the shape of the solution; the next chapter surveys the relevant literature, technologies and development methodologies that inform the design.

Chapter 2 — Literature Review / System Study

2.1 Evolution of Conversational AI and Chatbots

The idea of a machine that converses in natural language is as old as the field of Artificial Intelligence. The first widely known chatbot, ELIZA, was created by Joseph Weizenbaum in 1966 and simulated a psychotherapist through simple pattern-matching and scripted transformations of the user's input; it demonstrated that users readily attribute understanding to a system that merely reflects their words back, an observation now called the ELIZA effect. Progress followed through richer pattern languages such as AIML, and through the rise of statistical NLP. The commercial inflection point arrived with virtual assistants — Siri, Alexa, Google Assistant — which combined speech recognition, statistical intent classification and structured back-end actions. In parallel, task-oriented dialogue systems matured in research, formalizing the pipeline of understanding, dialogue-state tracking, policy and generation. Most recently, large pre-trained transformer language models have enabled open-domain conversation of unprecedented fluency. The present system draws on this lineage selectively: it adopts the disciplined task-oriented pipeline for reliability, while borrowing the fluency of Large Language Models for the final phrasing step only, so that account facts remain under deterministic control.

2.2 Review of Similar Systems

The application of AI to telecom and general customer support is an active area in both industry and research. Operators have deployed intent-routing bots that triage incoming queries to the correct department, frequently-asked-question retrieval systems that surface knowledge-base articles, and, increasingly, generative assistants layered over knowledge bases through retrieval-augmented generation. Commercial contact-centre platforms provide real-time sentiment monitoring, agent-assist suggestions that recommend responses to human agents, and automated after-call summarization. Research prototypes have explored churn prediction from usage and interaction features, next-best-action recommendation, and the automation of dispute resolution. Each of these addresses a slice of the support problem. The distinctive position of this project is its **integration**: rather than solving a single sub-problem, it combines identity validation, deep-learning understanding, grounded generation, knowledge-based recommendation, reinforcement-learning optimization and multi-agent orchestration into one coherent, runnable, observable system that remains fully local, reproducible and privacy-safe.

2.3 Natural Language Processing, Understanding and Representation

Natural Language Processing enables computers to process human language. A typical pipeline pre-processes text (tokenization, normalization, contraction expansion) and converts it into a numerical representation. Classical representations include the bag-of-words model and TF-IDF, which weights each term by how frequent it is in a document relative to how common it is across the corpus, emphasizing distinctive words; extending the vocabulary with bigrams lets short but meaningful phrases such as “too expensive” become single features. Dense representations — word embeddings and contextual transformer embeddings — capture semantic similarity but require large pre-trained models. Natural Language Understanding converts an utterance into a structured form through **intent classification** (a supervised multi-class text classification problem) and **entity extraction** (slot filling, commonly handled with rules or sequence models). This project uses TF-IDF with bigrams because it is transparent, fast and effective on the short, domain-specific utterances typical of support chat, and it pairs the learned classifiers with rule-based entity extraction.

2.4 Sentiment Analysis, Deep Learning and Language Generation

Sentiment analysis determines the affective polarity of text and, in a support setting, both modulates tone and triggers escalation. Techniques range from lexicon-based scoring (for example VADER) to supervised and transformer-based classifiers. Deep learning uses multi-layer neural networks to learn hierarchical representations; the Multi-Layer Perceptron is the foundational feed-forward architecture, trained by back-propagation, and even over TF-IDF features it captures interactions a linear model cannot. Recurrent networks and transformers model word order and long-range dependencies

and underpin modern language models. Natural Language Generation produces text from an internal representation; while Large Language Models generate fluent text, they can hallucinate — confidently stating false information — which is unacceptable in billing. This project therefore uses a grounded, hybrid generation strategy in which templates carry the authoritative facts and a local Large Language Model rephrases them under an explicit instruction not to invent anything, preserving fluency while eliminating hallucination.

2.5 Reinforcement Learning and Recommender Systems

Reinforcement Learning is a paradigm in which an agent learns to make decisions by interacting with an environment and receiving rewards, balancing exploration against exploitation. The multi-armed bandit is its simplest setting: an agent repeatedly chooses among several arms, each returning a stochastic reward, and seeks to maximize cumulative reward; the epsilon-greedy strategy explores a random arm with small probability and otherwise exploits the best-estimated arm. Recommender systems predict the items a user will value, classically through collaborative or content-based filtering; in a telecom offer context, however, eligibility is governed by hard business rules rather than taste, which argues for a knowledge-based recommender. This project combines a deterministic, rule-plus-scoring recommender with an epsilon-greedy bandit that learns which offers customers accept, giving recommendations that are both explainable and adaptive.

2.6 Dialogue Management and Multi-Agent Architectures

Dialogue management governs conversational flow: it tracks state, decides the next action, and maintains context. Task-oriented systems often model each task as a finite-state machine or a frame-filling process. As applications grow, a monolithic policy becomes unwieldy, motivating multi-agent designs in which specialized agents handle different sub-tasks. The hub-and-spoke (orchestrator) pattern is a widely used topology: a central coordinator receives every input, decides which specialized agent should handle it, and mediates all communication, so the specialized agents remain decoupled and never call one another directly. This yields clean separation of concerns, controlled hand-offs and easy extensibility, and it is the pattern adopted by the present system, augmented with a lightweight policy engine that converts detected signals into tone and escalation directives.

2.6a Natural Language Processing in Customer-Service Applications

Natural Language Processing has become central to customer-service technology, and understanding how it is applied in practice frames the design decisions of this project. The most common application is intent detection for routing, where an incoming message is classified so that it can be directed to the correct workflow or department; this is precisely the role the intent classifier plays here, distinguishing a bill-explanation from a dispute from an offer inquiry. A second application is sentiment and emotion monitoring, used both to prioritize distressed customers and to alert or coach human agents; the present system uses sentiment not only to observe but to act, allowing rising frustration to drive tone and escalation. A third is entity and information extraction, pulling structured details such as account numbers, dates and amounts from free text so that the system can act on them; this is the role of the entity extractor. A fourth, more recent application is generative assistance, where a language model drafts replies or summarizes conversations; the present system adopts this only for surface phrasing, under strict grounding, precisely because unconstrained generation is unsafe in a billing context. Seeing where each NLP capability fits in a real service pipeline clarifies why this project combines several of them — classification, sentiment, extraction and constrained generation — rather than relying on any one alone.

2.7 Software Development Models

The choice of a software-development life-cycle model shapes how a project is planned and executed. The **Waterfall** model proceeds through sequential phases — requirements, design, implementation, testing, deployment — and suits projects with stable, well-understood requirements, but it accommodates change poorly. The **Iterative and Incremental** model builds the system in repeated cycles, each delivering a working increment, allowing lessons from one cycle to

inform the next. **Agile** methodologies such as Scrum push this further with short time-boxed iterations, continuous feedback and embracing change. The **Spiral** model overlays risk analysis on iterative development. For this project an **iterative, incremental** model was chosen and organized around six milestones, because the machine-learning components required experimentation and refinement that a rigid Waterfall plan could not accommodate, while the bounded scope did not warrant the full ceremony of a formal Agile process. Each milestone functioned as a build–measure–learn cycle: a component was designed, implemented, verified through the offline scenario driver, and refined in light of what the verification revealed, before the next milestone began. This model is revisited in the implementation methodology of Chapter 5.

2.7a Why an Iterative Model Was Chosen

The selection of an iterative, incremental model over the alternatives deserves justification because it shaped how the entire project was executed. A strict Waterfall model was unsuitable because the machine-learning components could not be fully specified in advance: the intent taxonomy, the corpus size, the model hyper-parameters and the generation prompt all required empirical tuning, and Waterfall's assumption of stable, fully-understood requirements would have forced premature commitment to choices that only experimentation could settle. A full Agile process with its ceremonies and roles was, conversely, heavier than a single-developer, bounded-scope project warranted. The iterative model struck the right balance: it partitioned the work into six milestones aligned with the natural dependency order of a machine-learning application — data before preparation, preparation before modelling, modelling before generation, a working conversation before a learning loop, and a complete brain before deployment — and it treated each milestone as a build–measure–learn cycle whose measurement directly informed the next design decision. The numerical-instability, perspective-drift and rendering issues described later were all discovered and resolved within the milestone in which the relevant component was built, which is exactly the feedback-driven correction an iterative model is designed to enable. This model is therefore not an incidental choice but one that materially improved the quality and coherence of the delivered system.

2.8 Relevant Frameworks and Libraries

Several open-source frameworks and libraries underpin the implementation, and it is worth explaining how each relates to the system rather than merely restating its documentation. **Streamlit** is a Python framework for building interactive web applications; its native chat components (a message container and a fixed input box) provide exactly the conversational interface this project needs without a separate JavaScript front-end or build step. **scikit-learn** is a machine-learning library that supplies the TF-IDF vectorizer and the Multi-Layer-Perceptron classifier used for intent and sentiment, together with the utilities for train/test splitting and metric computation used in evaluation; it was chosen for its transparency and CPU efficiency. **Ollama** is a runtime that serves Large Language Models locally over a simple HTTP interface, which allows the optional natural-language phrasing to run offline with no API key and no data leaving the machine. **NumPy** and **pandas** support numerical and tabular operations, **joblib** serializes trained models, and **python-dotenv** loads configuration. Rather than dictating the architecture, these libraries are composed behind narrow interfaces so that any one of them — most notably the language-model provider — can be replaced without disturbing the rest of the system.

2.9 Comparative Analysis of Design Approaches

Three broad approaches to building a support chatbot can be contrasted, and the choices in this project are a direct response to their trade-offs, summarized in Table 2.1. The purely **rule-based** bot is predictable and transparent but brittle and unable to sense sentiment. The fully **generative** agent, driven by a single large model, is fluent but risks hallucinating facts, is hard to constrain to policy, and is opaque. The **hybrid, modular** approach adopted here uses learned models for understanding, deterministic logic for decisions, and a language model only for phrasing under strict grounding, thereby inheriting the reliability of the rule-based design where correctness matters and the fluency of the generative design where errors are harmless, while adding an adaptive learning loop that neither pure approach offers cheaply.

Criterion	Rule-based	Fully generative (LLM)	Hybrid (this project)
Fluency	Low	High	High (via optional LLM)
Factual reliability	High	Low (hallucination risk)	High (grounded)
Transparency	High	Low	High
Sentiment awareness	None	Implicit	Explicit
Adaptivity	None	Costly to retrain	Lightweight RL loop
Local / private	Yes	Often remote	Yes

Table 2.1 — Comparison of Design Approaches.

2.9a Text Representation in Depth

Between raw text and a classifier lies the crucial step of representation. The bag-of-words model records which words are present and how often, discarding order; despite its crudeness it is effective for topic-like classification. TF-IDF refines this by down-weighting words that appear in many documents and are therefore uninformative, such as “the” or “please,” and up-weighting words distinctive to a document; formally, term frequency measures how often a term occurs in a document, inverse document frequency measures how rare the term is across the corpus, and their product gives each term a weight capturing its discriminative value. Extending the vocabulary to include bigrams — pairs of adjacent words — allows short but meaningful phrases such as “too expensive” or “dispute ticket” to become single features, which materially improves classification of the phrase-heavy utterances typical of support chat. Dense representations take a different route: word embeddings map each word to a low-dimensional vector such that words used in similar contexts lie close together, and contextual embeddings from transformer models produce a different vector for a word depending on its context. These dense representations underpin modern deep NLP but require large pre-trained models and are harder to inspect. For a bounded, domain-specific classification task on short utterances, and for a proof of concept that prizes transparency and reproducibility, TF-IDF with bigrams offers an excellent balance of accuracy, speed and interpretability, which is why it is the representation chosen here.

2.9b Evaluation Metrics for Conversational Systems

Evaluating a conversational system requires metrics at several levels. At the component level, classification tasks are measured with accuracy, precision, recall and the F1-score, and a confusion matrix exposes which classes are mutually confused; because a single split can mislead on small corpora, stratified splitting and cross-validation give more stable estimates. At the conversation level, task-oriented systems are judged by task-completion rate — here the resolution rate — the number of turns to completion, and latency. At the experience level, user satisfaction is captured directly, for example through thumbs feedback yielding a satisfaction proxy, or inferred from sentiment trajectories. This project reports metrics at all three levels, in direct correspondence with the fifth objective, and the separation of the evaluation harness from the running application means these metrics can be regenerated on demand as the system evolves.

2.9c Justification of the Hybrid Choice

The comparative analysis leads directly to the architectural decision at the heart of this project. A billing assistant operates in a domain where an incorrect statement about money is not a minor blemish but a potential legal and financial liability, so factual reliability cannot be compromised; yet customers increasingly expect the natural, empathetic interaction that only fluent generation provides, so fluency cannot be ignored either. The rule-based approach secures the first at the cost of the second, and the fully generative approach secures the second at the risk of the first. The hybrid approach resolves the tension by assigning each concern to the mechanism best suited to it — deterministic logic for the facts and decisions, a language model for the surface phrasing — and by strictly forbidding the language model from touching the facts. This is not a compromise that sacrifices a little of each, but a separation of concerns that achieves both fully, and it is the single

most important design decision in the project. The remaining chapters can be read as the elaboration and validation of this decision.

2.10 Research Gap

Two gaps motivate the design. First, purely LLM-driven support agents, while fluent, are difficult to constrain and can hallucinate account facts — an unacceptable risk in billing. Second, purely rule-based bots are reliable but rigid and unnatural, and cannot learn. The gap is therefore a design that captures the fluency of generative models and the reliability of deterministic logic simultaneously, while remaining transparent and adaptive. This project addresses that gap through its hybrid, grounded, hub-and-spoke architecture: deterministic understanding and decision-making for correctness, an optional local language model for fluency, and a reinforcement-learning loop for adaptivity. The chapters that follow develop this design, beginning with the system analysis.

Chapter 3 — System Analysis

System analysis translates the problem and objectives of Chapter 1 into precise requirements and establishes that the proposed solution is feasible. This chapter states the functional, non-functional and user requirements, presents the feasibility study, and specifies the high-level system architecture, the data-flow diagrams and the use-case diagram that together define what the system must do.

3.1 Functional Requirements

ID	Functional Requirement
FR-1	Authenticate a customer via email and password, then verify identity via the last four digits of the ID on file before serving any account data.
FR-2	Classify each customer utterance into one of the defined intents and detect its sentiment.
FR-3	Extract entities (charge references, amounts, dispute topic, yes/no confirmations) from utterances.
FR-4	Explain the current bill and surface any unusual charge (roaming or overage).
FR-5	Present evidence for a disputed charge and offer tier-appropriate resolution options.
FR-6	Apply a provisional credit or goodwill waiver, or create a persisted dispute ticket, per the customer's choice.
FR-7	Compute personalized offer recommendations and pitch the highest-ranked eligible offer.
FR-8	Apply an accepted offer and record the acceptance as reinforcement-learning feedback.
FR-9	Hand off the conversation to the retention specialist on haggling, sustained negative sentiment, exhausted offers or explicit request.
FR-10	Run the retention negotiation ladder and apply accepted offers under a simulated supervisor approval.
FR-11	Answer ticket-status queries.
FR-12	Display live per-turn signals and aggregate KPIs, and accept thumbs feedback.

Table 3.1 — Functional Requirements.

3.2 Non-Functional Requirements

ID	Non-Functional Requirement
NFR-1 Performance	Deterministic per-turn processing within a few milliseconds; LLM-phrased replies within a few seconds.
NFR-2 Reliability	Degrade gracefully to templated responses if the LLM or any optional dependency is unavailable.
NFR-3 Correctness	Replies must never contain fabricated account facts; all figures derive from the data layer.
NFR-4 Usability	The interface requires no training and guides the customer through login, verification and chat.
NFR-5 Security/Privacy	Account data served only after identity verification; synthetic data used exclusively.
NFR-6 Maintainability	Modular business logic; data adapters and LLM provider swappable via configuration.
NFR-7 Observability	Every turn emits structured signals; sessions are logged for KPI aggregation.
NFR-8 Portability	Runs on commodity hardware with a single-command launch.

Table 3.2 — Non-Functional Requirements.

3.3 User Requirements

From the end-user's perspective, the customer requires the ability to sign in securely and prove their identity; to ask questions about their bill in their own words and receive clear, accurate explanations; to contest a charge and be shown the evidence behind it; to obtain an immediate remedy or a tracked dispute ticket; to be told about relevant offers and to accept

or decline them easily; to negotiate when an offer is unsatisfactory; and to reach a resolution without long waits or repetition. The operator, as a secondary stakeholder, requires that the agent apply policy consistently, never disclose data to an unverified party, never state incorrect financial figures, and produce a record of each interaction for monitoring and quality assurance. These user requirements are fully covered by the functional and non-functional requirements above.

3.4 Feasibility Study

3.4.1 Technical Feasibility

The system is technically feasible with widely available open-source tools. The NLU models train with scikit-learn on a CPU in seconds; the interface uses Streamlit; the optional language model is served locally by Ollama; and data is stored as JSON files, so no database server is required. The stack installs from a single requirements file and launches with one command, confirming that no specialized hardware or proprietary software is needed.

3.4.2 Economic Feasibility

Because the project relies exclusively on free and open-source software running on commodity hardware, its development cost is limited to time. In production, the economic case rests on deflecting a share of human-handled contacts to the automated agent, whose marginal cost per conversation is negligible relative to a staffed interaction, while simultaneously lifting revenue through better-targeted offers.

3.4.3 Operational Feasibility

Operationally, customers interact through a familiar, guided chat interface that needs no training, while the modular architecture lets the synthetic adapters be replaced with live connectors without changing the dialogue logic. The grounded generation and deterministic decision core make the system's behaviour auditable, which is essential for acceptance in a regulated billing context.

3.5 System Architecture

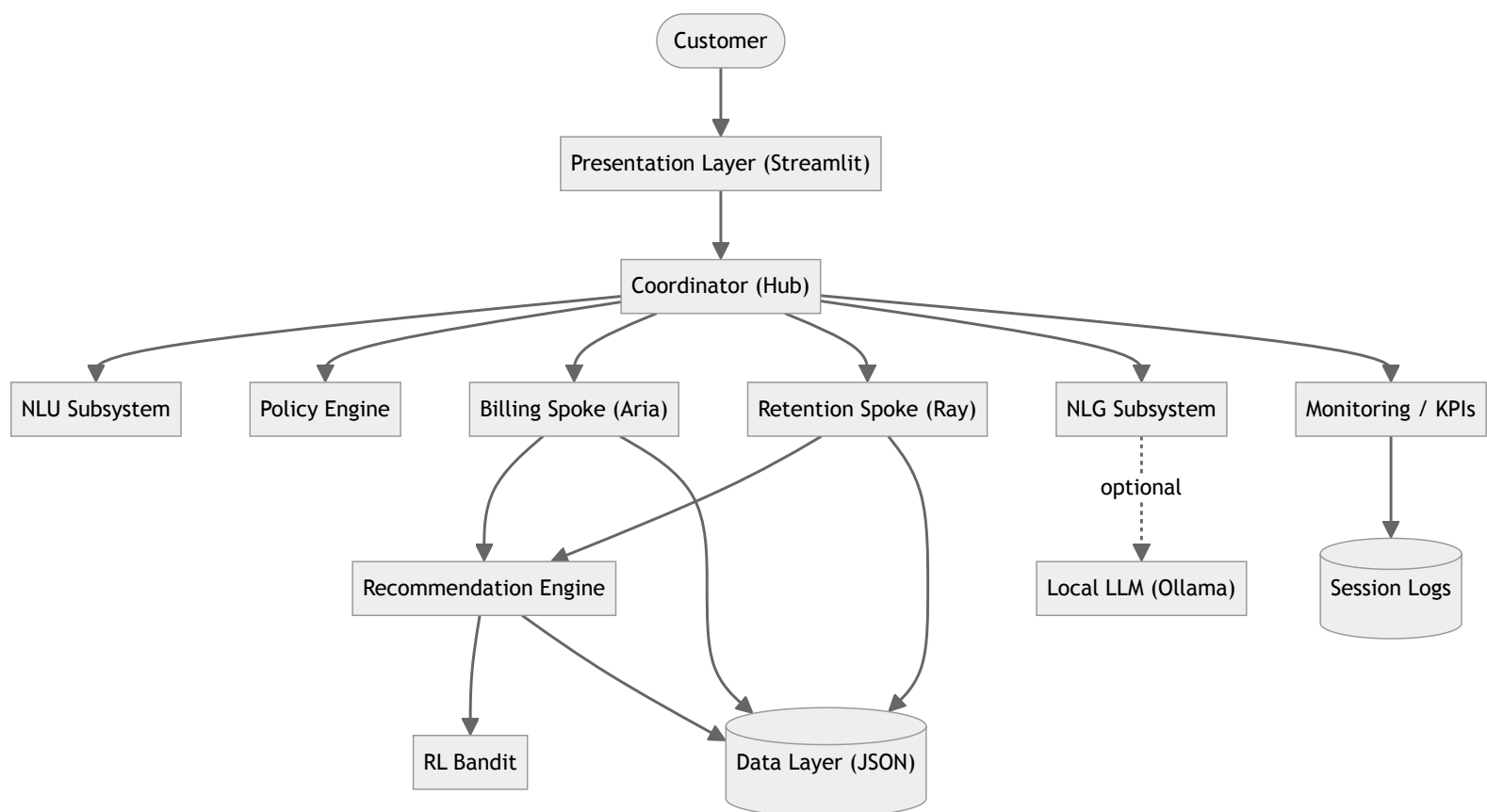


Figure 3.1 — System Architecture (high level). A layered design in which a coordinator orchestrates specialized subsystems over a shared data layer.

The architecture is layered. The presentation layer handles authentication and chat rendering; the coordinator is the hub that orchestrates each turn; specialized subsystems (NLU, policy, the two agent spokes, recommendation, reinforcement learning and generation) sit beneath it over a shared data layer; and a cross-cutting monitoring concern captures signals and KPIs.

3.5a Proposed System Workflow

The proposed system's workflow can be traced end-to-end to make the architecture concrete. A customer arrives at the welcome screen, proceeds to log in, and passes the identity-verification gate; only then is a conversation session created and the customer admitted to the chat. From that point the workflow is a loop: the customer sends a message; the message enters the coordinator, which invokes the understanding subsystem to produce a structured frame, consults the policy engine for a tone-and-escalation directive, and either routes the message to the currently active agent's flow or, if a hand-off condition is met, transfers control to the retention specialist; the selected flow acts on the account data — explaining a bill, presenting evidence, applying a remedy, or recommending an offer — and returns a grounded template with the relevant facts; the generation subsystem turns this into a natural-language reply; and the reply, together with the turn's signals, is returned to the interface and rendered, while the turn is recorded on the shared state. This loop repeats until the interaction reaches a resolved state, at which point the session may be saved for monitoring. Every step in this workflow corresponds to a subsystem in the architecture of Figure 3.1 and to a process in the data-flow diagrams that follow, so the analysis, the architecture and the eventual implementation remain in exact correspondence.

3.6 Data-Flow Diagrams

The context-level (Level 0) data-flow diagram treats the whole chatbot as a single process exchanging data with the customer, the data store and the optional language model.

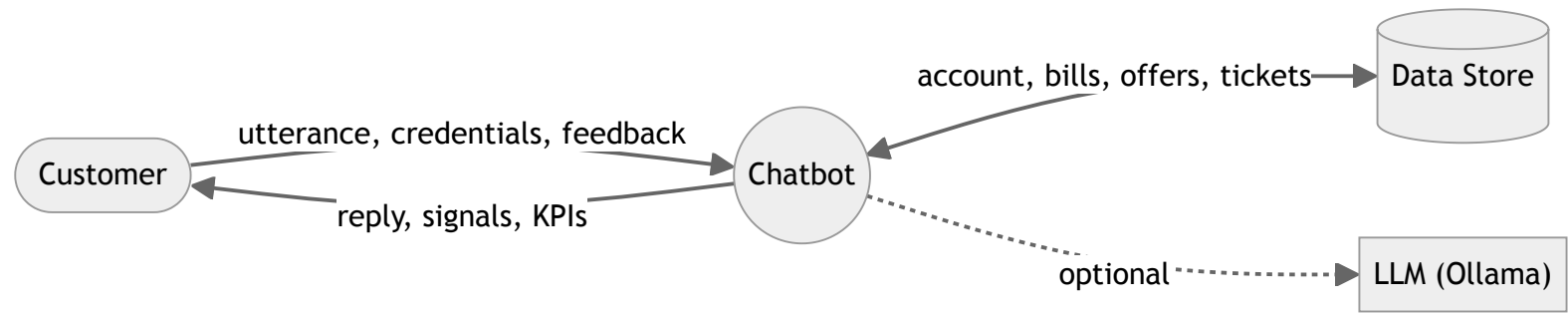


Figure 3.2 — Data-Flow Diagram, Level 0 (Context Diagram).

Explanation. At the context level the entire chatbot is a single process. The customer supplies utterances, credentials and feedback, and receives replies, live signals and KPI summaries in return. The process exchanges account data — customers, bills, offers and tickets — with the data store in both directions, reading to answer queries and writing when a ticket is created or an offer applied. The optional language model is shown as a one-way call used only for phrasing. This diagram fixes the system boundary and its external interfaces before the internal processes are elaborated.

The Level 1 diagram expands the chatbot into its principal processes and the data stores they read and write.

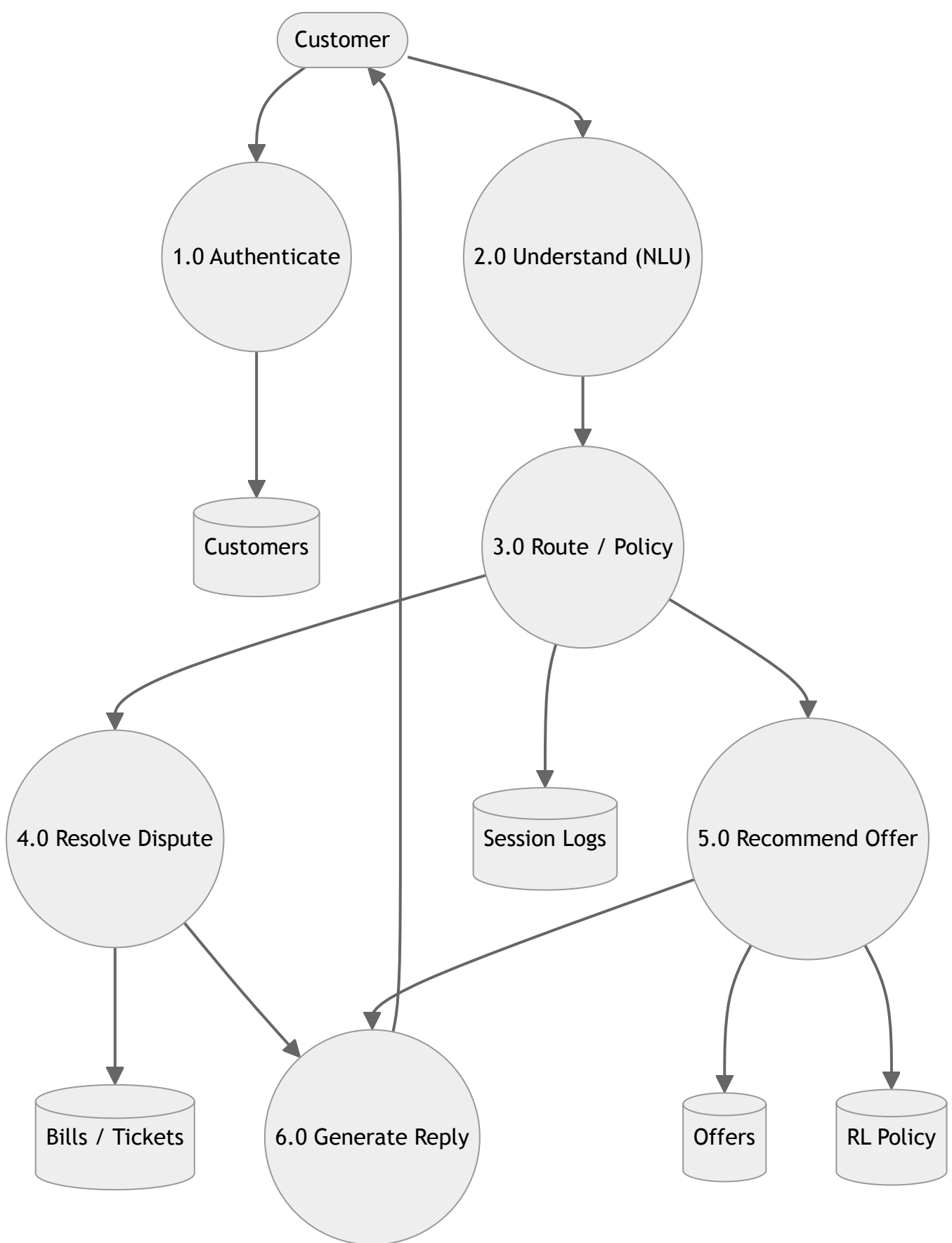


Figure 3.3 — Data-Flow Diagram, Level 1.

Explanation. The Level 1 diagram decomposes the single process into six numbered sub-processes: authentication, understanding, routing and policy, dispute resolution, offer recommendation and reply generation. It also shows the data stores each reads or writes — the customer store for authentication, the bills and tickets store for dispute resolution, the offers store and the reinforcement-learning policy for recommendation, and the session-log store written by the routing process. The flows between processes trace how a single utterance moves from understanding through routing to either dispute resolution or recommendation and finally to generation before returning to the customer, giving a process-level view that maps directly onto the subsystems implemented in the system.

3.7 Use-Case Diagram

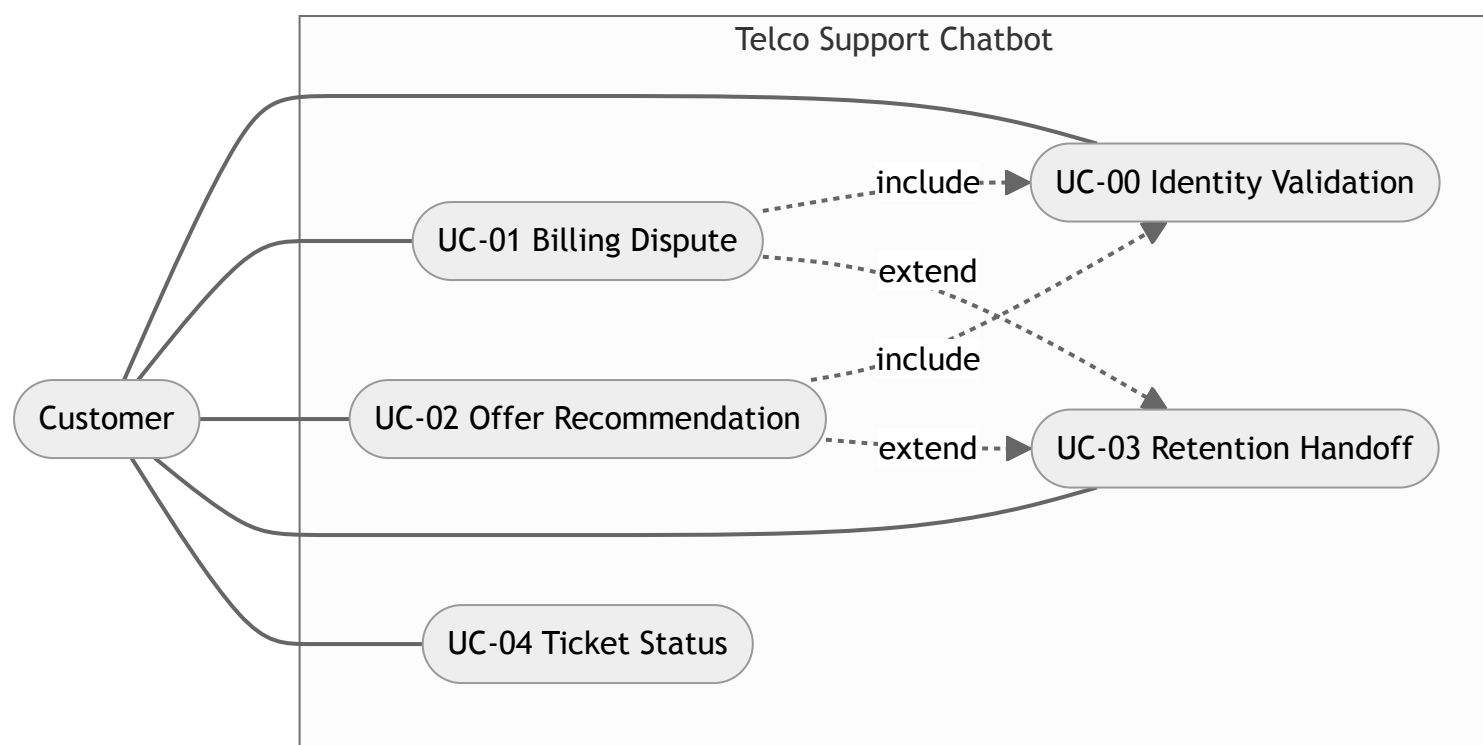


Figure 3.4 — Use-Case Diagram. Billing and offer use cases include identity validation and may extend into a retention handoff.

The single external actor is the Customer. Internally, the AI agents Aria and Ray, the recommendation engine and the reinforcement-learning component participate. The five use cases are realized in detail, with sequence diagrams, in Chapter 4.

3.8 Assumptions and Constraints

The analysis proceeds under explicit assumptions and constraints that bound the problem. It is assumed that the customer interacts in written English through a browser, that the account, bill, offer and policy data required to answer a query are available through the data-access layer, and that a customer can be authenticated by a combination of login credentials and a knowledge factor. The principal constraints are that the system must run on commodity hardware without a graphics processor, must not depend on any paid or remote service to function, must protect account data behind authentication, and must never present fabricated financial information. These constraints are not incidental; they directly motivate the choice of lightweight neural classifiers over large models, the optional and locally hosted language model, the mandatory identity-validation step, and the grounded-generation discipline developed in Chapter 4.

3.9 Risk Analysis

A brief risk analysis informed several design decisions. The most serious risk in a billing domain is the disclosure of incorrect financial information; this is mitigated by grounding all figures in the data layer and forbidding the language model from inventing facts. A second risk is unauthorized disclosure of account data, mitigated by the identity-validation gate. A third is dependence on an external service; mitigated by making the language model optional with an automatic template fallback. A fourth is model error on unfamiliar phrasings; mitigated by the keyword fallback and by keeping consequential decisions in deterministic logic rather than in a learned model. By identifying these risks during analysis, the design could address each structurally rather than reactively.

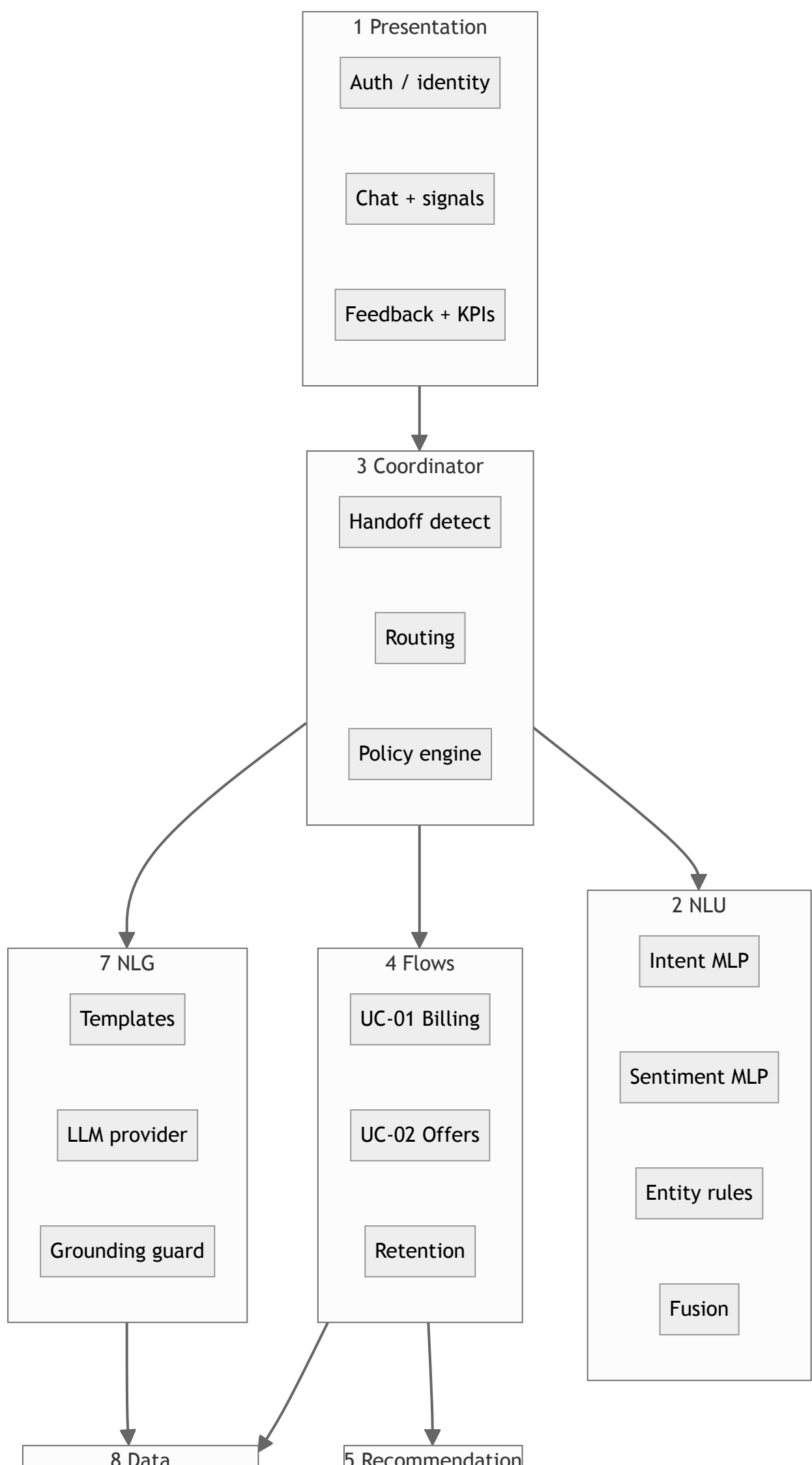
Chapter 4 — System Design

This chapter presents the detailed design of the system through Unified Modeling Language and related diagrams — subsystem decomposition, class diagram, activity diagrams, sequence diagrams and an entity-relationship diagram — together with the data schema and table structures. Every diagram is numbered and followed by an explanation, as required.

4.1 Design Goals and Principles

The design is governed by four principles. **Thick brain, thin interface:** all understanding, decision-making and action live in the Python brain while the interface only renders. **Deterministic core, optional intelligence:** facts and decisions are computed by rules and data, while the language model is confined to phrasing. **Graceful degradation:** every dependency has a fallback so the system never hard-fails. **Observability by design:** each turn emits structured signals. These principles reconcile the competing demands of fluency and reliability identified as the research gap in Chapter 2.

4.2 Subsystem Decomposition



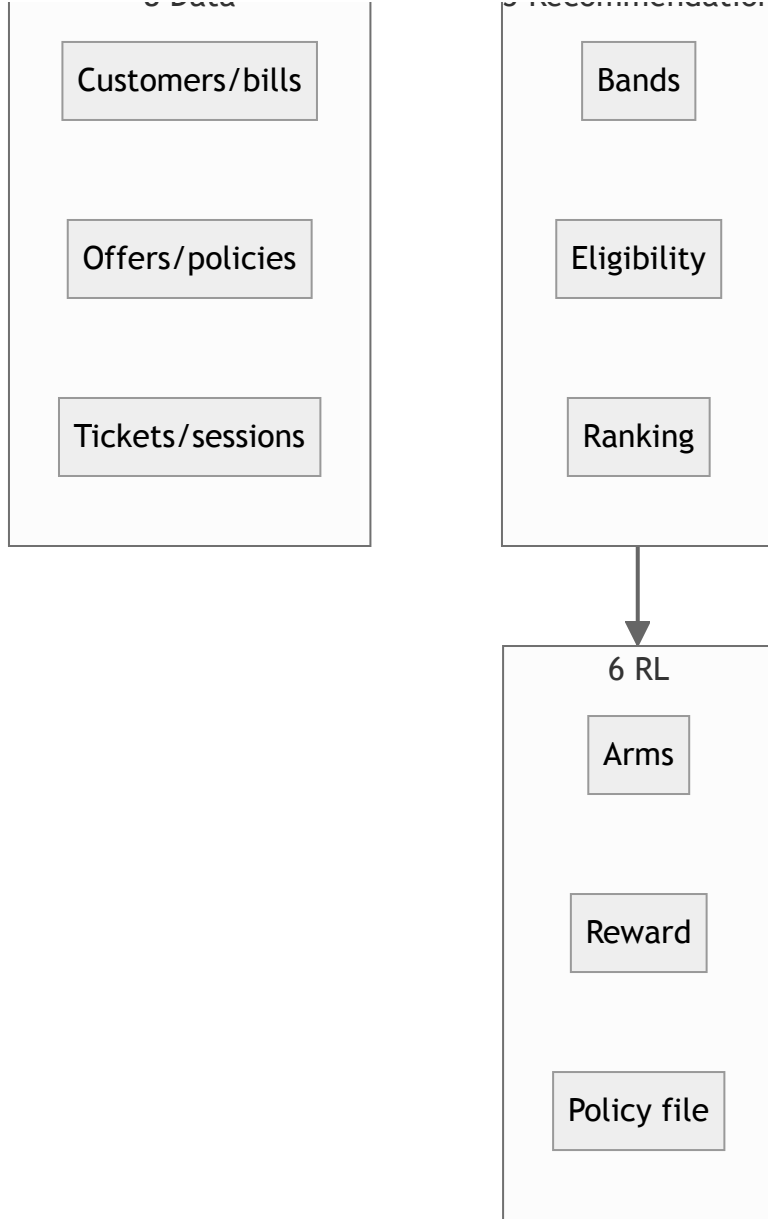
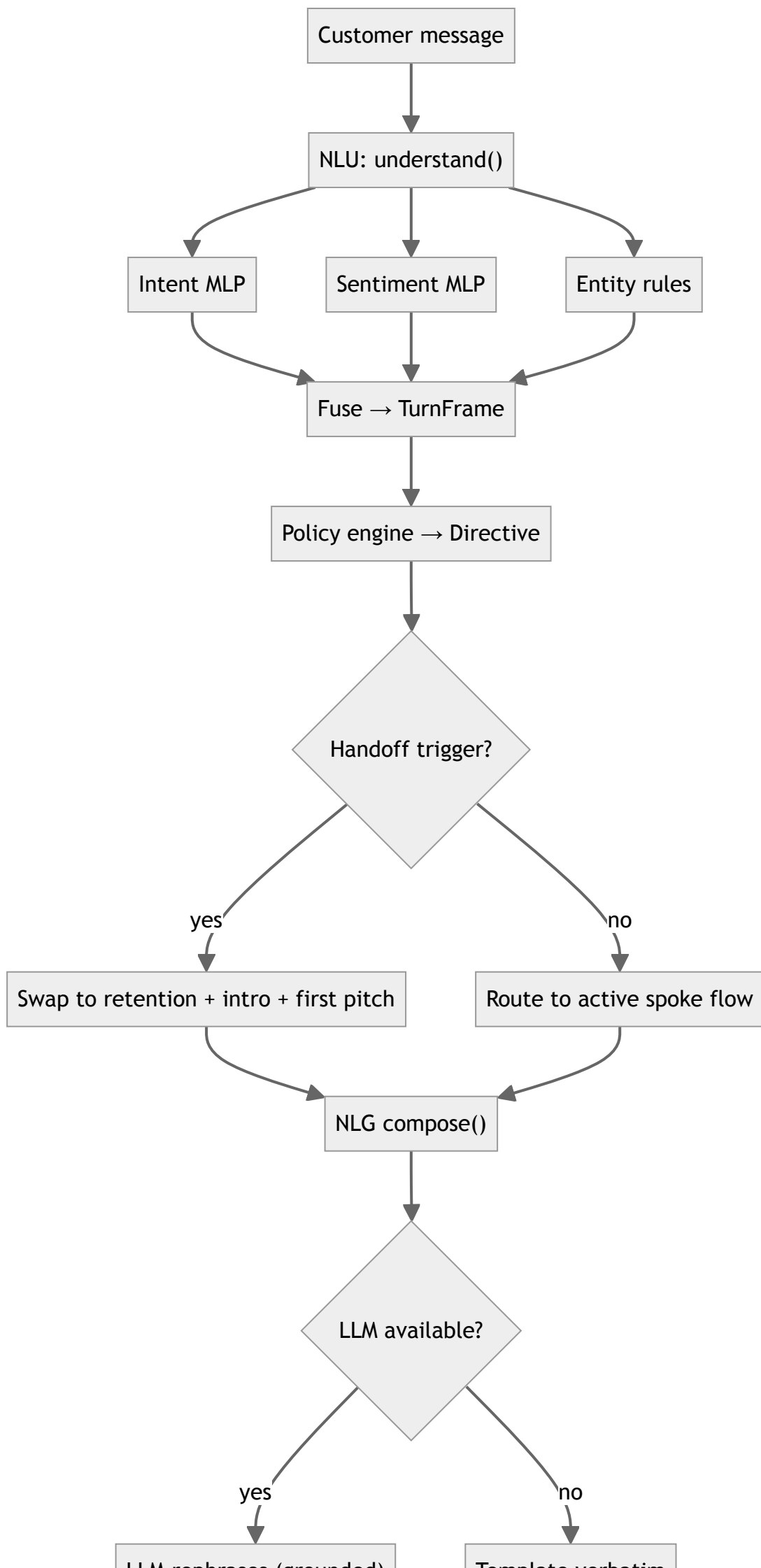


Figure 4.1 — Subsystem Decomposition (eight cohesive subsystems).

#	Subsystem	Responsibility	Modules
1	Presentation	Auth, chat, signals, KPIs, feedback	ui/streamlit_app.py, app/auth.py
2	NLU	Intent + sentiment + entities → TurnFrame	app/nlu/*
3	Coordinator	Hub routing, handoff, policy	app/coordinator.py, policy_engine.py
4	Flows	Use-case state machines	app/flows/*
5	Recommendation	Bands, eligibility, scoring, ranking	app/recommendation_engine.py
6	RL	Bandit arms, reward, persisted policy	app/rl/bandit.py
7	NLG	Templates, LLM provider, grounding guard	app/nlg/*
8	Data	Access to accounts, offers, tickets, logs	app/data_store.py, data/*.json

Table 4.1 — Subsystem Responsibilities.

4.3 Per-Turn Processing Pipeline



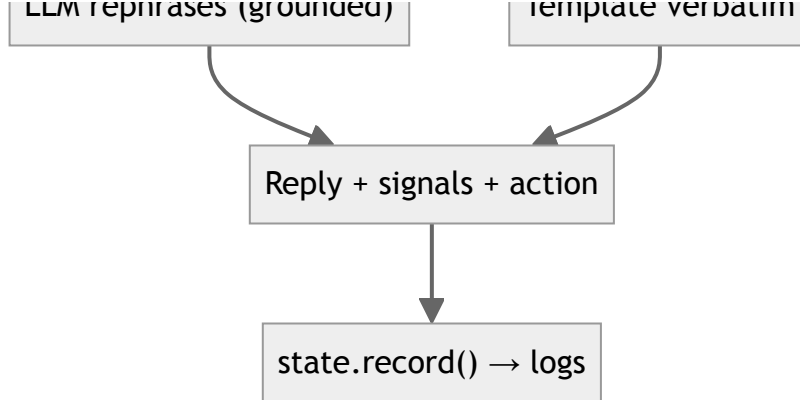


Figure 4.2 — Per-Turn Processing Pipeline (the deterministic control flow executed for every message).

4.4 Hub-and-Spoke Structure



Figure 4.3 — Hub-and-Spoke Structure. The two AI spokes communicate only through the hub.

4.5 Class Diagram

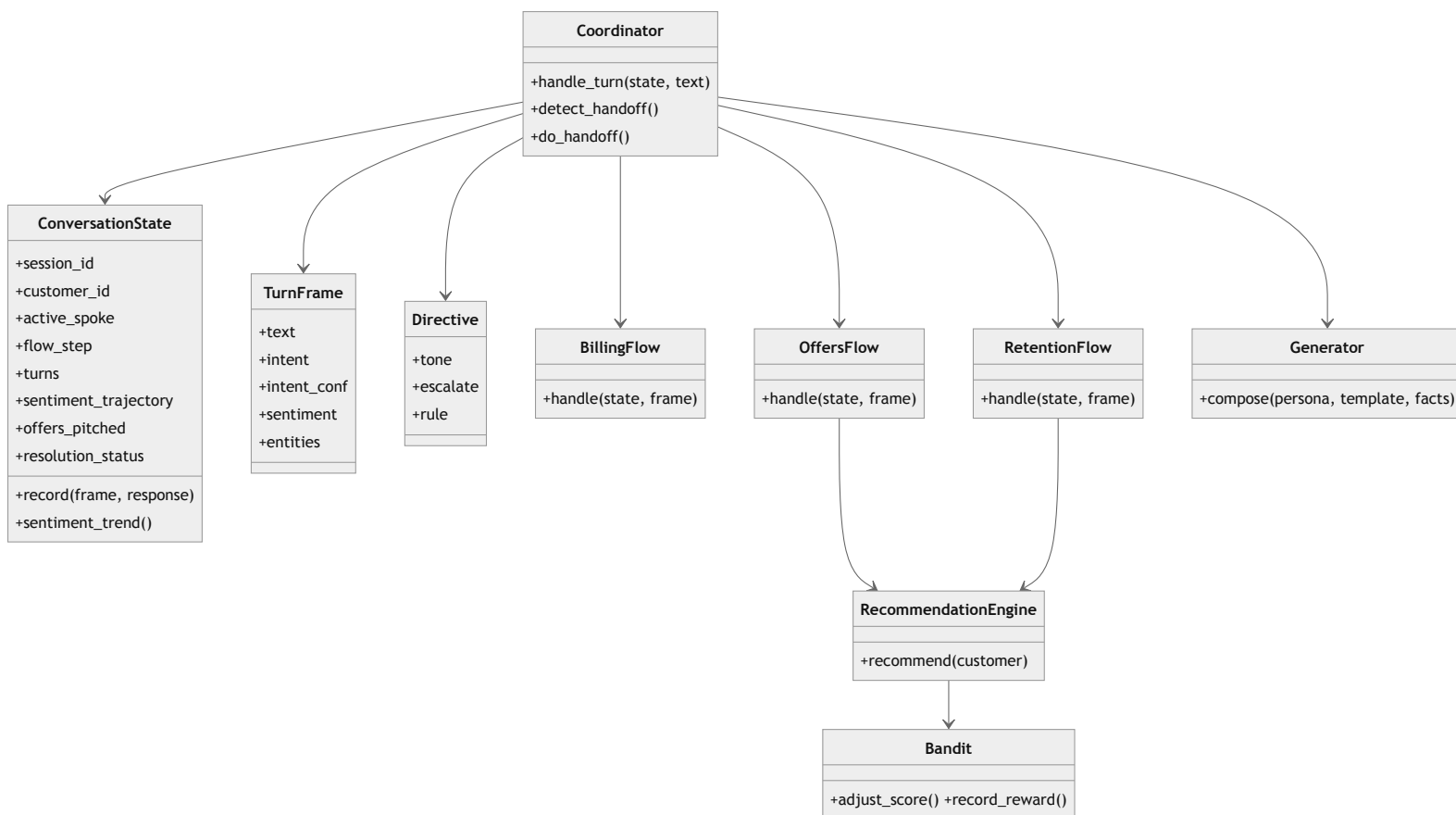


Figure 4.4 — Class Diagram. The Coordinator orchestrates the shared ConversationState and the flow, recommendation, learning and generation classes.

The design favours cohesive classes and modules with narrow interfaces over deep inheritance hierarchies. ConversationState is the single shared object that every component reads and writes; TurnFrame and Directive are lightweight value objects produced by the NLU and policy stages; and the flow, recommendation, bandit and generator classes each encapsulate one responsibility.

4.6 Entity-Relationship Diagram

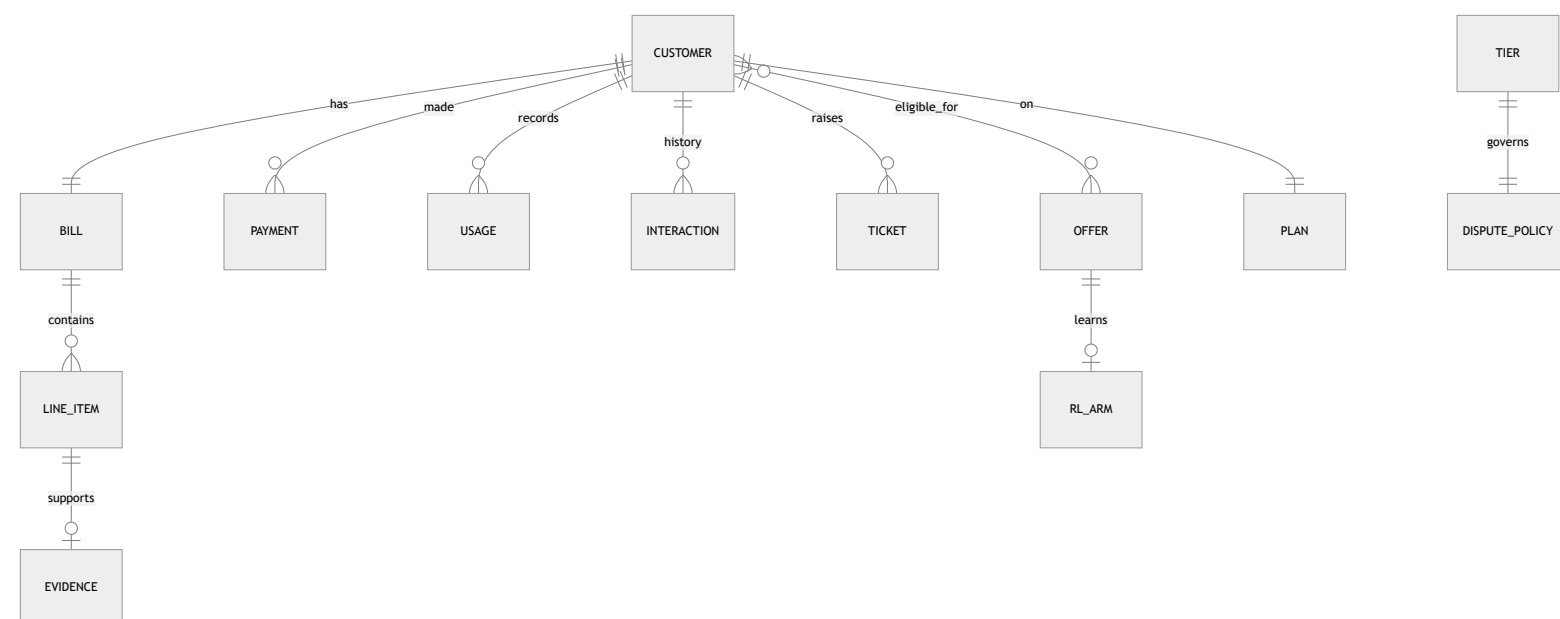


Figure 4.5 — Entity-Relationship Diagram of the synthetic data model.

The central entity is the Customer, associated with one Bill and one Plan and with many Payments, Usage records, Interactions and Tickets. A Bill comprises Line Items, some carrying Evidence. Offers relate to Customers through eligibility and to reinforcement-learning arms; dispute policy is keyed by loyalty Tier.

4.7 Data Schema and Table Structures

Although the proof of concept stores data as JSON documents rather than in a relational database, the data is fully structured and can be described as a set of logical tables. Table 4.2 gives the schema of the principal entities; a relational deployment would map these directly onto database tables.

Entity (table)	Key fields
Customer	customer_id (PK), first_name, last_name, email, phone, plan (FK), tenure_months, arpu_usd, loyalty_tier, segment, last4_id
Bill	bill_id (PK), customer_id (FK), cycle, total_usd, previous_usd, due_date, status
LineItem	charge_id (PK), bill_id (FK), description, amount_usd, type, evidence
Payment	txn_id (PK), customer_id (FK), date, amount_usd, method, status
Offer	offer_id (PK), title, target_tier, min_tenure_months, discount_pct, value_usd, applies_to
Plan	plan_id (PK), monthly_usd, data_cap_gb, lines_included, tier_fit
Ticket	ticket_id (PK), customer_id (FK), subject, status, category, charge_id (FK), sla_hours, created_ts
DisputePolicy	tier (PK), eligible, max_usd, auto_approve, sla_hours
RLArm	offer_id (PK/FK), shows, accepts

Table 4.2 — Data Schema (logical table structures; PK = primary key, FK = foreign key).

4.8 Activity Diagrams

Activity diagrams model the flow of control within a use case. Figure 4.6 shows the billing-dispute activity and Figure 4.7 the offer-recommendation activity including the reinforcement-learning feedback.

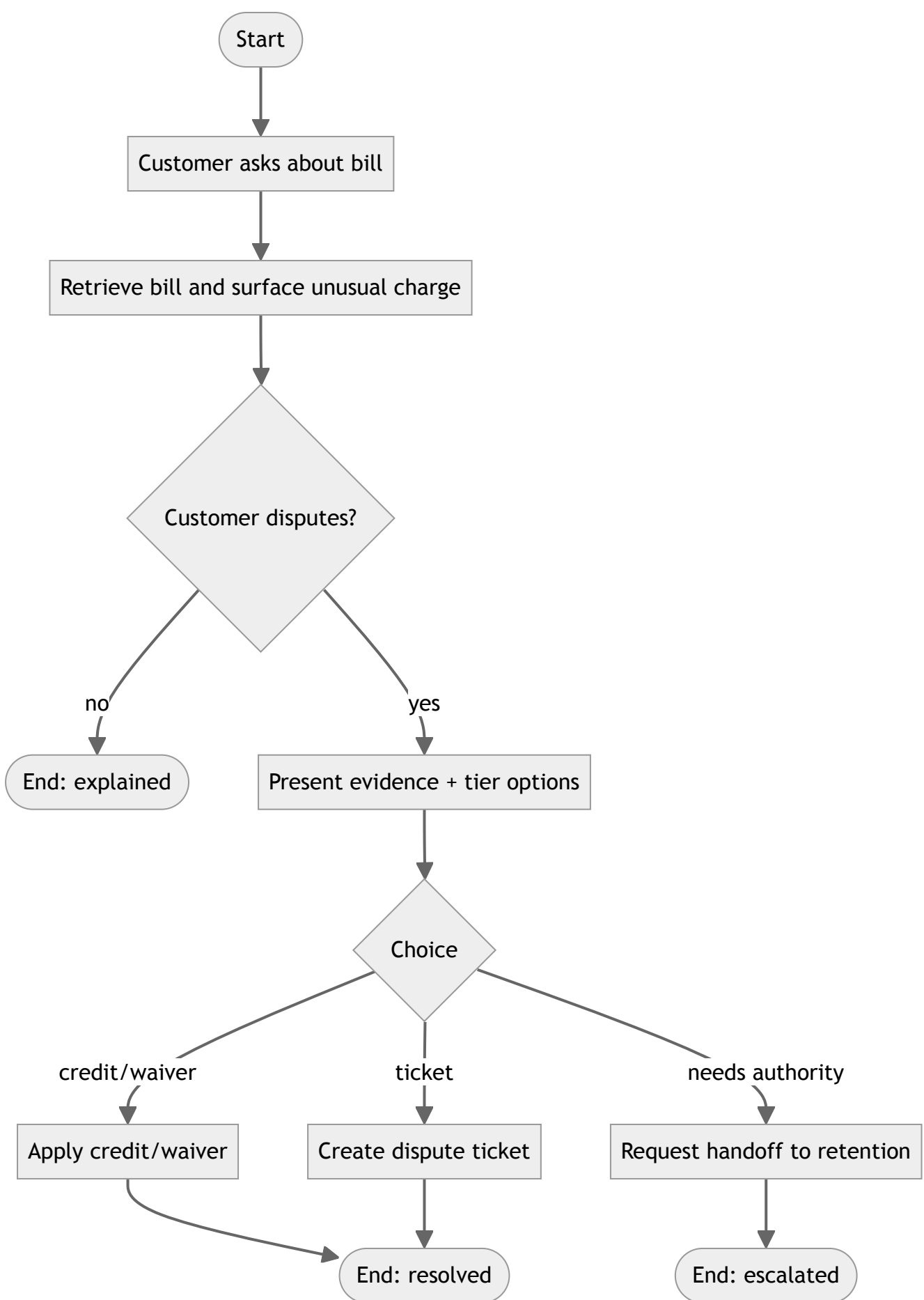


Figure 4.6 — Activity Diagram: Billing Dispute Resolution.

Explanation. The activity begins when the customer raises a bill question and the system retrieves the bill and surfaces the unusual charge. A decision node then branches on whether the customer disputes the charge: if not, the activity ends with the charge simply explained. If the charge is disputed, the system presents the evidence and the tier-appropriate options, and a second decision node branches on the customer's choice — applying a credit or waiver, creating a dispute ticket, or, when the required action exceeds the front-line agent's authority, requesting a hand-off to retention. Each terminal state

corresponds to a distinct resolution outcome, and the diagram makes explicit that a dispute always terminates in one of three well-defined ways rather than being left open.

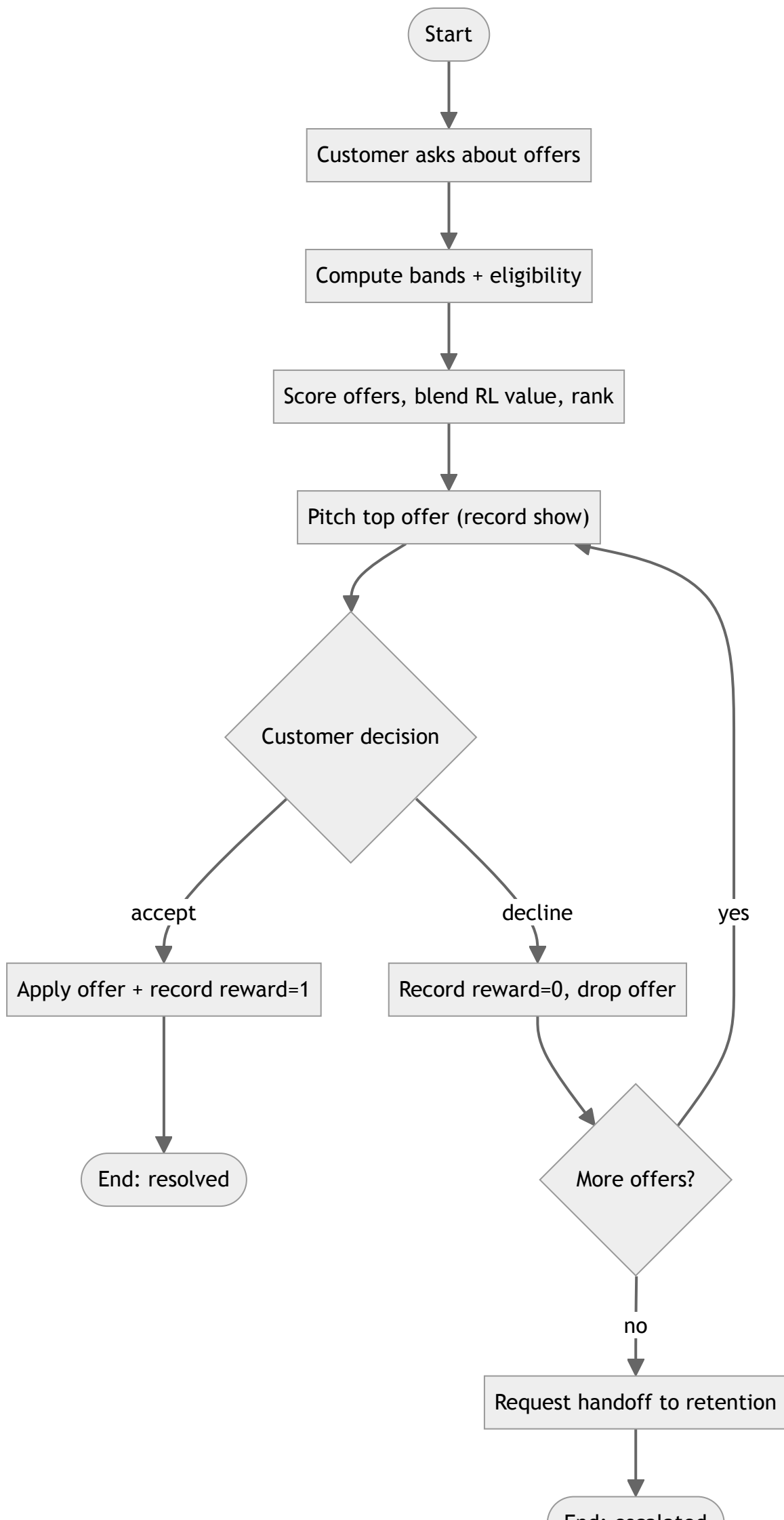


Figure 4.7 — Activity Diagram: Offer Recommendation with Reinforcement Learning.

Explanation. The activity begins when the customer asks about offers; the system computes the customer's bands and eligibility, scores the eligible offers, blends in the learned reinforcement-learning value, and ranks them. It then pitches the top offer and records that it was shown. A decision node branches on the customer's response: an acceptance applies the offer and records a reward of one, ending the activity resolved; a decline records a reward of zero, drops the offer, and loops back to pitch the next-best offer. A second decision node handles exhaustion — when no offers remain, the activity requests a hand-off to retention rather than ending empty-handed. The loop and the reward-recording steps make the feedback nature of the process explicit, distinguishing this activity from a one-shot recommendation.

4.9 Sequence Diagrams (Use-Case Realizations)

Each use case is realized as an interaction among the customer, the coordinator, the agents and the data/recommendation subsystems. Figures 4.8 to 4.12 present the five sequence diagrams; the accompanying narrative for each use case appears in Chapter 5's module descriptions and in the walkthrough of Section 5.9.

4.9.1 UC-00 Identity Validation

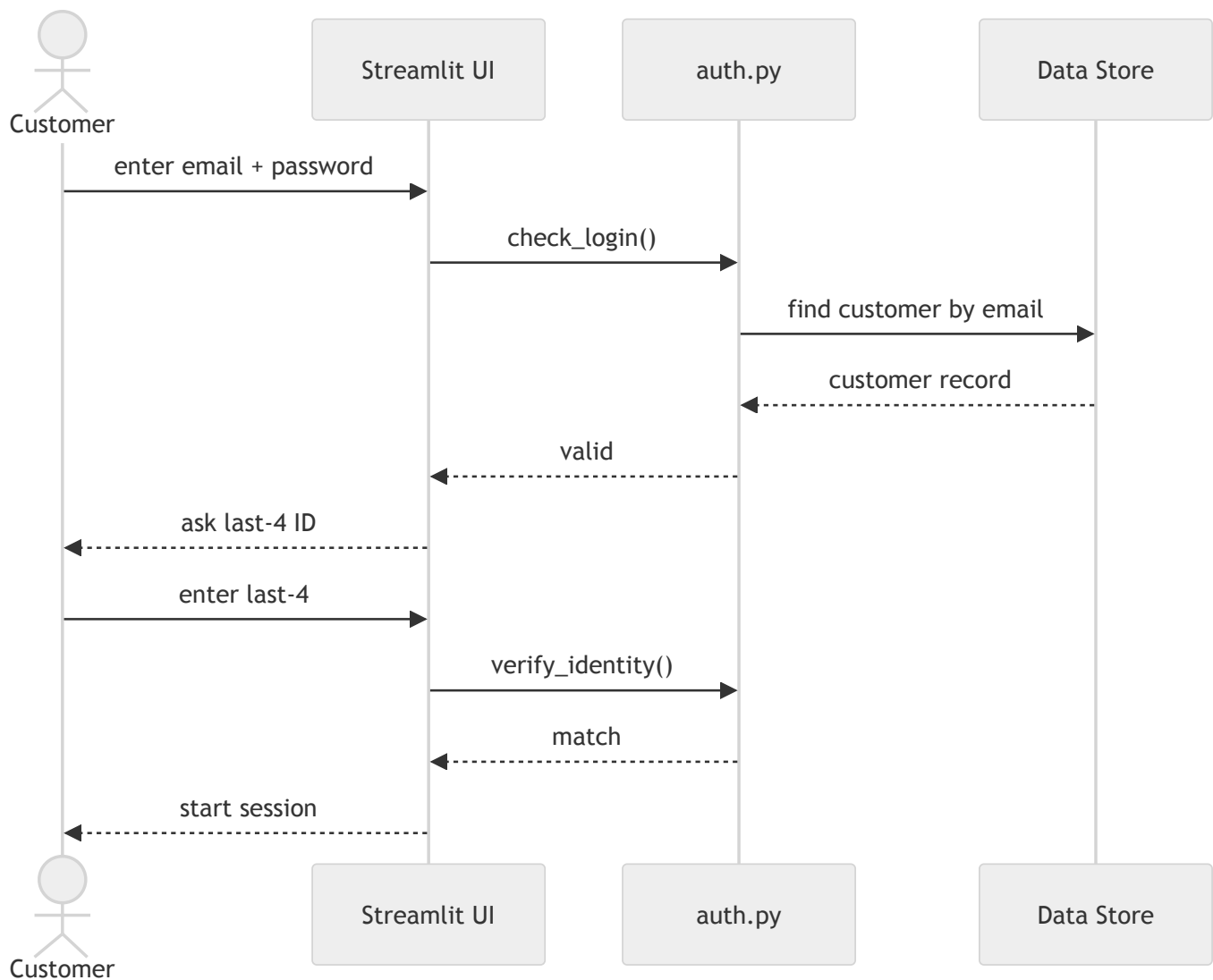


Figure 4.8 — Sequence Diagram: UC-00 Identity Validation.

Explanation. The customer first submits an email and password, which the interface passes to the authentication component; that component looks the customer up in the data store and confirms the credentials. Only after this first factor succeeds does the interface request the second factor — the last four digits of the identity on file — and compare it with the stored value. The session, and therefore access to any account data, is created only when both factors match. This two-step

interaction realizes functional requirement FR-1 and the privacy requirement NFR-5, and its explicit sequencing guarantees that no account information can be reached before identity is established.

4.9.2 UC-01 Billing Dispute

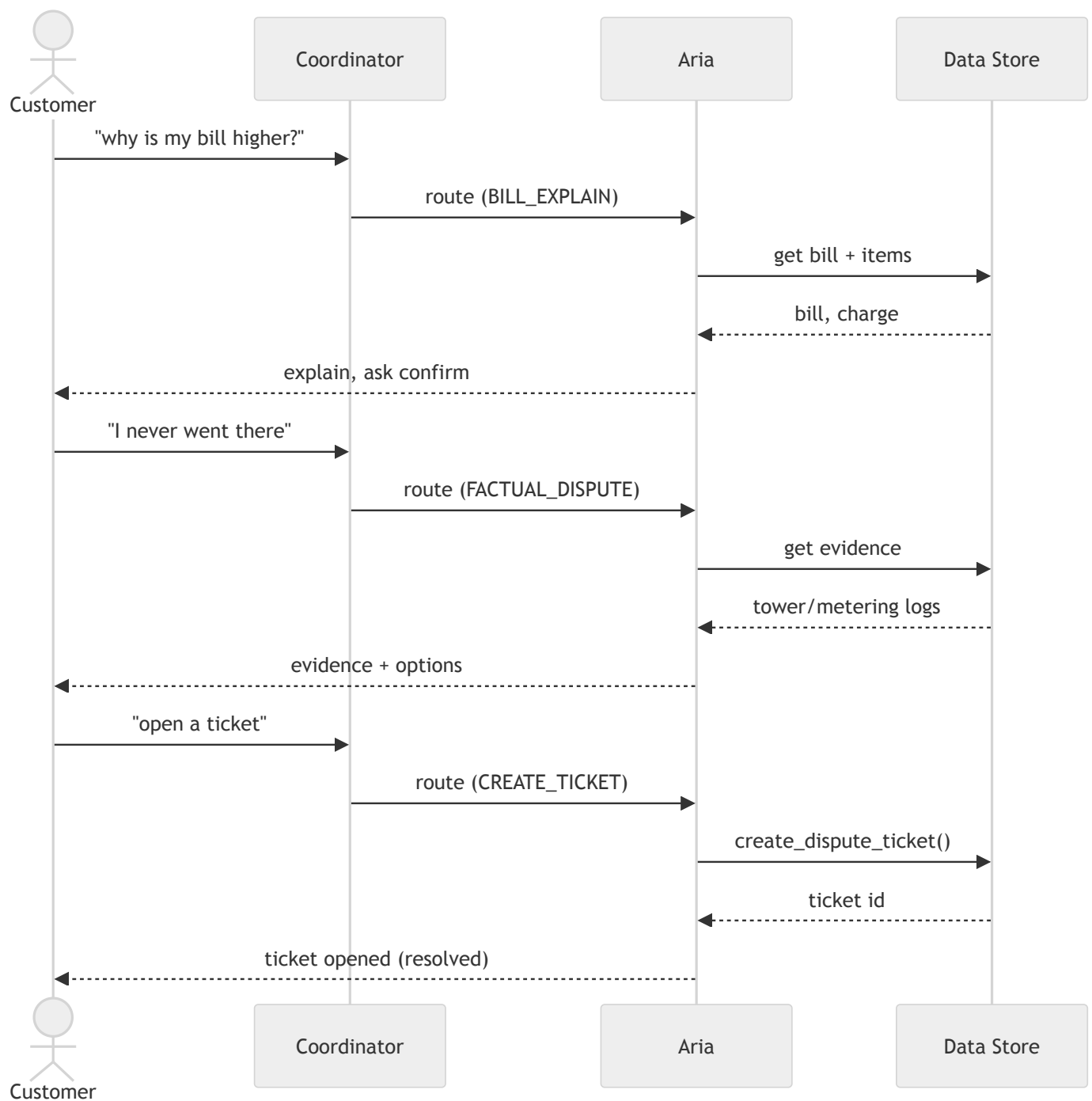


Figure 4.9 — Sequence Diagram: UC-01 Billing Dispute Resolution.

Explanation. The interaction unfolds over several turns mediated entirely by the coordinator. On the first turn the coordinator routes a bill-explanation intent to the billing agent, which retrieves the bill and its line items and replies by surfacing the unusual charge. On the next turn a factual dispute causes the agent to fetch the charge evidence and present it together with the tier-appropriate resolution options. On the final turn the customer chooses to open a ticket, and the agent creates a persisted dispute ticket and returns its identifier. Each message from the agent is grounded in data fetched from the store during that turn, and the progression of flow steps within the billing agent is what allows the same agent to behave differently on each successive turn.

4.9.3 UC-02 Offer Recommendation

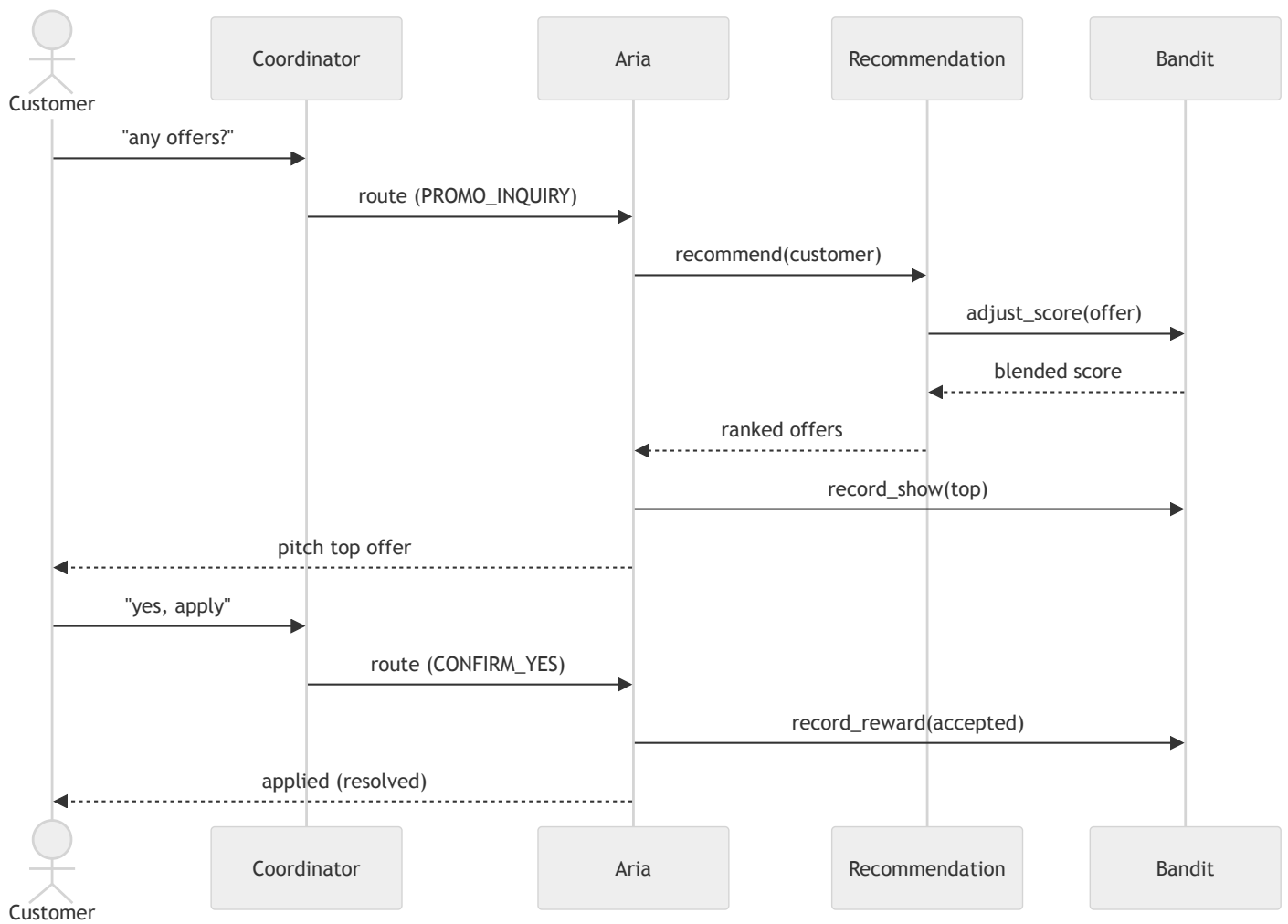


Figure 4.10 — Sequence Diagram: UC-02 Offer Recommendation.

Explanation. When the customer inquires about offers, the coordinator routes the turn to the billing agent, which asks the recommendation engine for a ranking. The engine, in turn, consults the reinforcement-learning bandit to blend each offer's learned acceptance value into its deterministic score before returning the ranked list. The agent records that the top offer was shown — a signal the bandit later uses — and pitches it. On acceptance, the agent records a positive reward for that offer's arm and applies it. This sequence shows the recommendation and learning subsystems cooperating within a single turn, and it is the point at which the system's adaptivity is exercised.

4.9.4 UC-03 Retention Handoff

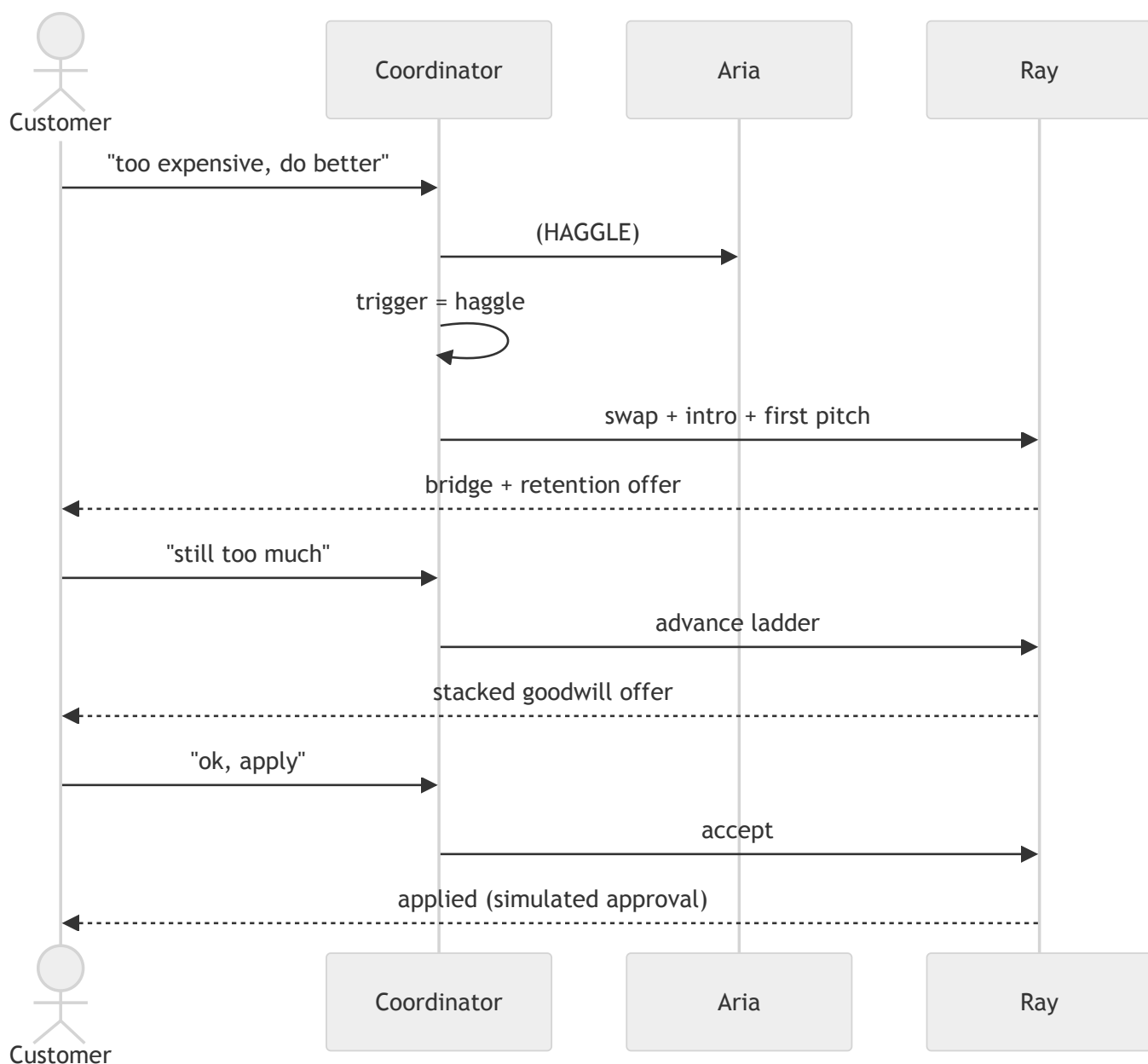


Figure 4.11 — Sequence Diagram: UC-03 Retention Handoff and Negotiation.

Explanation. The interaction begins on the billing spoke, where a haggling intent causes the coordinator's hand-off detector to fire. The coordinator then swaps the active spoke to retention and, in the same turn, produces the specialist's bridging opening and first ladder rung — the customer experiences a single, smooth transition rather than an awkward pause. Over subsequent turns the specialist climbs the ladder, and on acceptance applies the offer under a simulated supervisor approval. Note that the billing and retention agents never exchange messages directly; every transfer of control passes through the coordinator, which is the defining property of the hub-and-spoke design and what keeps the two agents decoupled.

4.9.5 UC-04 Ticket Status

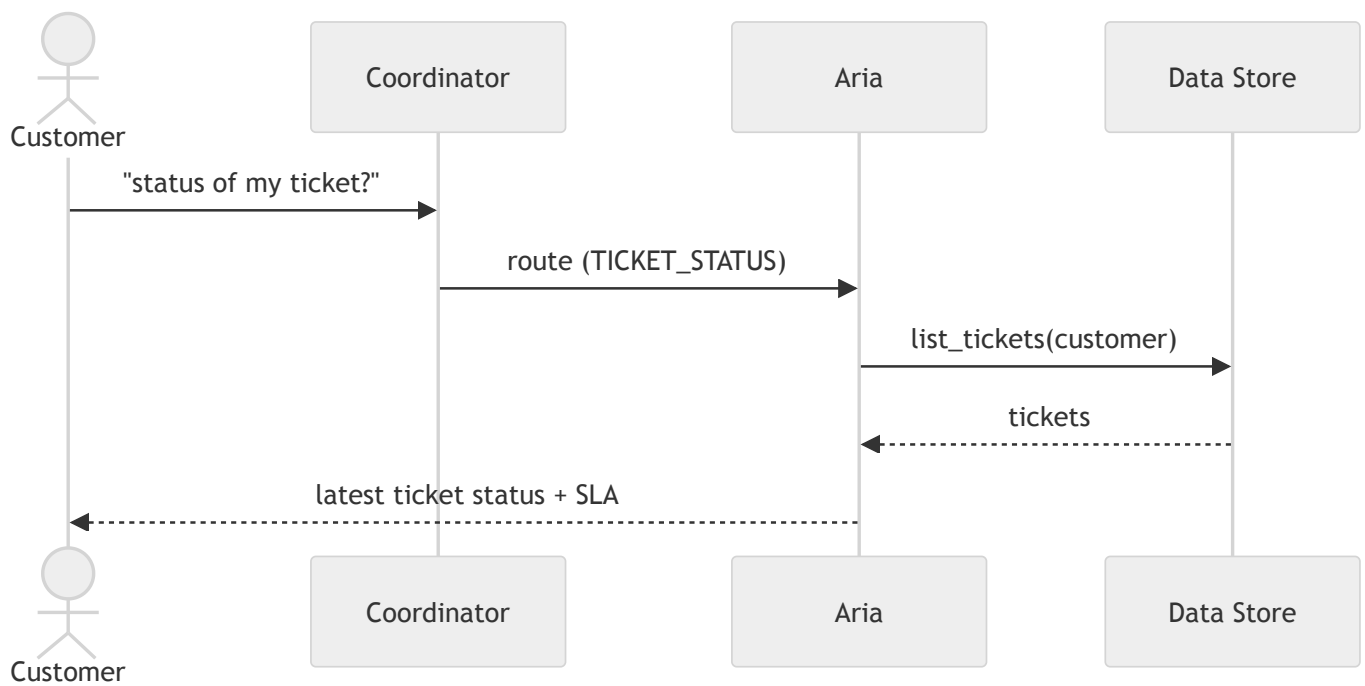


Figure 4.12 — Sequence Diagram: UC-04 Ticket Status Query.

Explanation. This is the simplest interaction. A ticket-status intent is routed to the billing agent, which reads the customer's tickets from the store and returns the most recent ticket's identifier, status and service-level window in a single turn. Although short, it completes the ticket lifecycle begun in UC-01 — creation followed later by status retrieval — and it demonstrates a durable read over the persisted ticket store, complementing the write demonstrated when a ticket is opened.

4.10 User-Interface Design

The interface is designed as a linear screen flow — welcome, login, identity verification, chat — that guides the customer with minimal cognitive load. The chat screen places the conversation centrally, with a persistent sidebar presenting the customer's context, live per-turn signals, the reinforcement-learning table and the KPI panel. Each agent persona has a distinct avatar so that a hand-off is visually obvious. The concrete rendered screens, which serve as the UI wireframes realized in the working application, are presented as screenshots in Chapters 5 and 7.

4.11 Conversation-State Design

All per-turn components read from and write to a single conversation-state object, which is the linchpin of the design. Centralizing state in one serializable object avoids hidden coupling between components — no component holds private state that another cannot see — and it makes the entire conversation reconstructable from a single record. The object holds the session and customer identifiers; the active spoke and the reason for any hand-off; the current flow and step within it; a free-form slot dictionary used by the flows to remember in-progress details such as the charge under dispute or the offer currently pitched; the ordered list of completed turns; the intent and sentiment trajectories that the policy engine inspects; the sets of offers pitched, declined and accepted; the list of audited actions; the escalation flags; and the overall resolution status. Two derived operations are defined on it: appending a turn (which also updates the trajectories) and computing the sentiment trend over a window (which returns “declining” when the recent scores are strictly falling and currently negative). This design means that adding a new signal to track, or a new flow, requires only extending this object and the components that use it, without disturbing the rest of the system.

4.12 Design Patterns Employed

Several well-known design patterns structure the implementation. The **orchestrator** (hub-and-spoke) pattern governs the multi-agent topology, with the coordinator as the single mediator. The **strategy** pattern appears in the language-model

provider, where a common interface dispatches to interchangeable Ollama, OpenAI or Anthropic implementations selected at configuration time. The **state** pattern is realized in the dialogue flows, each of which is a small finite-state machine advancing a step field. The **adapter** pattern is embodied in the data-access layer, which presents a stable interface over the synthetic JSON store and could equally front a live back-end. The **template method** idea underlies the grounded generation, where a fixed template defines the content and an optional model varies only the surface form. Employing recognized patterns keeps the design comprehensible to another engineer and makes each extension point explicit.

4.13 Error Handling and Graceful Degradation

Robustness is a first-class design concern, expressed as graceful degradation at every external boundary. If the trained NLU models are absent, the intent classifier falls back to a keyword rule set and the sentiment analyser to a lexical scorer, so the system still functions before training. If the language model is unreachable or errors, the generator silently returns the deterministic template, so a network hiccup or a stopped Ollama service never breaks a conversation. If the recommendation engine finds no eligible offers, the flow requests a hand-off rather than failing. Every applied action is recorded with an explicit success status, so a partial failure is visible rather than silent. This design ensures that the worst-case behaviour is a slightly less fluent but still correct interaction, never an error presented to the customer.

4.14 Interface Contracts Between Subsystems

The subsystems communicate through small, well-defined contracts. The NLU subsystem exposes a single function that maps text to a turn frame. The policy engine maps a turn frame and the state to a directive. Each flow exposes a handler that takes the state and the frame and returns a response dictionary containing a template, a facts dictionary and optional extras such as offer cards or a hand-off request, while mutating the state in place. The generator maps a persona, a template, a facts dictionary and a directive to a final string. The recommendation engine maps a customer to a ranked list. These narrow contracts are what allow the subsystems to be developed, tested and replaced independently, and they are the practical embodiment of the maintainability non-functional requirement.

4.15 Scalability and Extensibility of the Design

Although a proof of concept, the design was shaped so that it could scale and be extended without structural change. Extensibility is achieved through the subsystem boundaries: a new intent requires only a few labelled examples and, if it needs a new behaviour, a new branch in a flow; a new offer requires only a catalogue entry, after which the recommendation engine and the bandit handle it automatically; a new agent persona could be added as a third spoke, and because the spokes communicate only through the hub, the existing spokes would be unaffected; and a new language-model provider is a single additional case in the provider abstraction. Scalability, in a production sense, is anticipated by the stateless nature of the per-turn processing — each turn is a pure function of the message and the conversation state — which means many conversations could be served concurrently by replicating the brain behind a load balancer, with the shared state externalized to a fast store. The synthetic data-access layer is the single seam at which a scalable, live back-end would be attached. In short, the boundaries drawn during design are precisely the boundaries along which the system would later grow.

Chapter 5 — System Implementation

This chapter describes how the design of Chapter 4 was realized in working software: the development environment and tools, the hardware and software requirements, the iterative methodology and its milestone outcomes, the module-wise implementation with key algorithms and representative code snippets, application screenshots, an end-to-end walkthrough, and the use of version control.

5.0 Implementation Overview and Development Workflow

The implementation translated each design element of Chapter 4 into working Python, subsystem by subsystem, following the milestone order. The development workflow for each subsystem was consistent: its interface contract was fixed first, so that the subsystem could be exercised in isolation; a minimal version was implemented to satisfy that contract; it was verified against crafted inputs or, for a flow, against a scripted conversation; and it was then refined in light of what the verification revealed before being integrated behind the coordinator. Because the coordinator depends only on the narrow contracts and not on the internals of each subsystem, subsystems could be built and improved independently and in parallel with the growth of the test suite. Configuration was externalized from the first milestone so that no experiment required a code edit, and the code was committed to version control at each meaningful step so that the history records the incremental construction of the system. This disciplined, contract-first workflow is what allowed a system spanning data engineering, machine learning, generation, reinforcement learning and interface design to be assembled without the components becoming entangled, and it is reflected in the clean, one-to-one correspondence between the subsystems of the design and the folders of the code base.

5.1 Development Environment

Development was carried out on a macOS workstation with an Apple-Silicon (arm64) processor, using Python 3.9 within an isolated virtual environment created with `venv`. Source code was edited in Visual Studio Code, and the application was run and tested locally from the terminal. The optional Large Language Model was served by a local Ollama installation. No cloud services, graphics processors or paid tools were required at any stage, which keeps the environment reproducible on any comparable laptop.

5.2 Tools and Technologies Used

Concern	Technology	Role in the system
Language / runtime	Python 3.9+	All application logic
User interface	Streamlit	Chat UI, signal panel, KPI panel
Machine learning	scikit-learn	TF-IDF + MLP intent and sentiment models, evaluation
Numerics / data	NumPy, pandas	Vector and tabular operations
Model persistence	joblib	Serialize trained models
Language model	Ollama (llama3.2)	Optional grounded rephrasing
Configuration	python-dotenv	Provider/model settings from <code>.env</code>
Data store	JSON files	Customers, bills, offers, tickets, logs
Version control	Git / GitHub	Source management and submission link

5.2a Rationale for the Technology Choices

Each technology was chosen deliberately, and the reasoning is worth making explicit because it reflects the project's guiding priorities of transparency, reproducibility and local operation. **Python** was selected as the single implementation language because it hosts the richest ecosystem for machine learning and natural language processing and allows³⁸ the brain

and the interface to share one language, eliminating the friction of a polyglot stack. **Streamlit** was chosen for the interface because it provides purpose-built chat components and turns a Python script directly into an interactive web application with no separate front-end build, no JavaScript and no web-server configuration, which is ideal for a proof of concept that must be launched with a single command and demonstrated on a laptop. **scikit-learn** was preferred over a heavy deep-learning framework because its Multi-Layer-Perceptron trains in seconds on a CPU, requires no graphics processor, and produces models that are easy to inspect and to evaluate with the same library's metric utilities; this satisfies the deep-learning objective while remaining lightweight. **Ollama** was chosen to serve the optional language model because it runs models entirely locally behind a simple HTTP interface, so the natural-language phrasing needs no API key, incurs no per-call cost, and keeps all data on the machine, which is important both for privacy and for offline demonstration. Finally, **JSON files** were chosen over a database server because the data volume is small, the files are human-readable and version-controllable, and they impose zero setup burden, while the data-access layer preserves the option of substituting a real database later. Collectively these choices minimize external dependencies and maximize the ease with which the system can be reproduced and inspected.

5.3 Hardware and Software Requirements

Category	Requirement
Processor	Modern multi-core CPU (Apple Silicon / x86-64); no GPU required
Memory	8 GB RAM minimum (16 GB recommended with the local LLM)
Disk	~2 GB environment + ~2 GB for the LLM model
Operating system	macOS or Linux (Windows via WSL)
Runtime	Python 3.9+ and pip
Optional	Ollama runtime with the llama3.2 model

Table 5.1 — Hardware and Software Requirements.

5.4 Development Methodology and Milestone Outcomes

An iterative, incremental methodology (Section 2.7) was executed across six milestones, each a build–measure–learn cycle whose output became the tested foundation of the next. Git underpinned the process, and configuration was externalized so that experiments required no code changes. Table 5.2 records the outcome of each milestone.

Milestone	Work performed	Outcome
M1 Data Collection	Curated 3 synthetic customers with bills, evidence, payments, usage, interactions, offers, policies and tickets	Complete self-contained dataset (data/*.json) and access layer
M2 Data Preparation	Hand-labelled 14-intent and 3-class sentiment corpora; shared text pre-processing	Reproducible labelled corpus (app/nlu/dataset.py)
M3 NLU Model	Trained TF-IDF+MLP intent and sentiment models; regex entities; fusion	Persisted NLU pipeline; ~83% sentiment, ~62% intent held-out
M4 NLG Module	Grounded templates + pluggable LLM provider with fallback	Fluent, hallucination-free replies (app/nlg/*)
M5 Reinforcement Learning	Epsilon-greedy bandit; reward from accept/decline; persisted policy	Adaptive offer ranking (app/rl/bandit.py)
M6 Deployment & Monitoring	Streamlit app; live signals; KPI aggregation; session logs	Single-command runnable, monitored application

Table 5.2 — Milestone Outcomes.

5.5 Module-wise Implementation

5.5.1 NLU Module

The intent classifier is a scikit-learn pipeline chaining a TF-IDF vectorizer (unigrams and bigrams) with a Multi-Layer-Perceptron classifier of two hidden layers and L2 regularization. At inference, the utterance is normalized, vectorized and classified; when the top class probability falls below a confidence threshold, the classifier defers to a keyword rule set to avoid confident errors on very short inputs. The sentiment analyser follows the same recipe and additionally returns a signed score computed as the probability-weighted sum of class polarities. Entity extraction uses regular expressions to identify charge identifiers, amounts, the dispute topic and yes/no confirmations. The three outputs are fused into a `TurnFrame`.

5.5.1a Entity Extraction Detail

Entity extraction is deliberately implemented with regular expressions and keyword matching rather than a learned sequence model, because the entities of interest in this domain are few, well-structured and reliably recognizable by pattern. Charge and offer identifiers follow fixed prefixes and are matched directly; monetary amounts are matched by a pattern that accepts an optional currency symbol followed by digits; the dispute topic is inferred by scanning the normalized text for topic keywords grouped under roaming, overage, tax and plan; and a yes/no confirmation is detected from affirmative or negative cue words. The extracted slots are attached to the turn frame and consumed by the flows — for instance, the billing flow uses the detected topic to select the right evidence, and the offers flow uses the affirmation to decide whether an offer was accepted. This rule-based approach is transparent, requires no training data, and is trivial to extend with a new pattern, which suits a bounded domain where precision on a small set of entity types matters more than open-ended recognition.

5.5.2 Recommendation Module

The recommendation engine derives the customer's churn, tenure and usage bands, filters offers by tier and minimum-tenure eligibility, scores each eligible offer with a weighted heuristic, passes the score through the bandit for learned re-weighting, and returns a ranked list, each recommendation carrying a human-readable reason. The bands are computed from concrete thresholds: the churn proxy is derived from the interaction history, treating a customer with an escalated prior interaction or with two or more prior contacts as high risk; the tenure band buckets tenure into high above three years, medium from seven to thirty-six months, and low below; and the usage band buckets recent data consumption into high above ninety percent of cap, medium above sixty percent, and low below. The weighted score begins from a base value and is increased for higher churn risk, for a data add-on when usage is high, for an exact tier match, and for a waiver when an overage charge is present, before being clamped to the unit interval. Eligibility encodes real business rules — an offer's target tier must match the customer's tier or be open to any tier, the customer's tenure must meet the offer's minimum, and the Bronze overage waiver is suppressed unless an overage charge actually exists — so that the engine never recommends an offer the customer cannot receive. The stated reason accompanying each recommendation is generated from the offer type and the customer's bands, giving every recommendation an explanation that the agent can voice, which is essential to the explainability the design values.

5.5.3 Reinforcement-Learning Module

The bandit maintains, per offer, counts of shows and accepts in a JSON policy file, estimates each offer's value as its acceptance ratio with an optimistic prior, blends that value into the deterministic score, and updates the counts as offers are shown and accepted or declined, persisting after each update.

5.5.4 NLG Module

The generator returns the template verbatim when no live LLM is configured; otherwise it builds a constrained system prompt and calls the provider abstraction, which dispatches to Ollama, OpenAI or Anthropic and returns the rephrased text or `None` on failure, in which case the template is used. The prompt forbids the model from introducing any new facts and instructs it to speak as the agent to the customer.

5.5.4a Ticketing and Persistence

Dispute tickets are the one class of state that persists beyond a single session, and they are handled by a small ticketing capability within the data layer. When the billing flow decides to open a formal dispute, it constructs a ticket record carrying a freshly generated identifier, the customer and charge references, a subject, an open status, a category, a service-level-agreement window of forty-eight hours, and a creation timestamp, and it appends this record to the persisted ticket store on disk. Ticket-status queries later read this same store and report the most recent ticket's identifier, status and window. This demonstrates a complete create-and-read lifecycle over durable storage, and it is the natural seam at which a production system would substitute a real ticketing back-end for the JSON file without altering the flow logic that produces and consumes tickets.

5.5.5 Coordinator and Flows

The coordinator's `handle_turn` runs the NLU pipeline, evaluates the policy engine, routes to the billing or offers flow while checking hand-off triggers (swapping to retention when one fires), composes the reply, records the turn with its latency, and returns a structured response. The flows are small state machines that read and mutate the shared state and append audited action records.

5.5.6 Presentation and Configuration

The Streamlit application is a screen state machine (welcome, login, verify, chat) driven by session state; the chat screen renders messages with per-persona avatars, a hand-off banner, ranked offer cards, per-turn signal captions and feedback buttons. Dollar signs in reply text are escaped before rendering so the framework does not misinterpret them as mathematics. Configuration is read from `.env`, allowing the LLM provider to be changed without editing code.

5.5.7 The Coordinator Response Object

Each call to the coordinator returns a single structured response object that the interface consumes, and its shape is worth describing because it is the contract between the brain and the presentation. The object carries the speaking persona and the final reply text; the active spoke and, if a hand-off occurred on this turn, a flag and the hand-off reason; the detected intent and its confidence, and the sentiment label and score; the policy rule that fired; any offer cards to display; the most recent audited action, if one was taken; the resolution status; and the measured latency. By returning all of this in one object, the coordinator lets the interface render not only the reply but the full set of live signals without reaching into the brain's internals, which preserves the thin-interface principle. The same object, minus the presentation-only fields, is what is appended to the conversation state and ultimately serialized to the session log, so the signals shown live and the signals logged for evaluation are one and the same, guaranteeing that the monitoring reflects exactly what the customer experienced.

5.6 Key Algorithms

Two algorithms are central to the system's behaviour. The first ranks offers by combining a rule-based score with the learned bandit value; the second updates the bandit from feedback. They are given below in pseudocode.

```
Algorithm 1: Recommend offers for a customer
Input: customer c, catalogue O
1. bands ← compute_bands(c)           // churn, tenure, usage
```

```

2. scored ← []
3. for each offer o in O:
4.     if eligible(o, c, bands):
5.         base ← weighted_score(o, c, bands)      // in [0,1]
6.         q ← bandit.value(o)                    // accepts/shows
7.         final ← 0.7*base + 0.3*q                // blend
8.         scored.append((o, final, reason(o,c)))
9. sort scored by final descending
10. return top-N of scored

```

Algorithm 2: Update bandit from feedback

Input: offer o, accepted (bool)

```

1. arm ← policy[o] (default {shows:0, accepts:0})
2. if accepted: arm.accepts ← arm.accepts + 1
3. persist policy to disk

```

5.7 Important Code Snippets

The per-turn entry point ties the subsystems together (full source is in Appendix B):

```

def handle_turn(state, text, use_llm=True):
    frame = pipeline.understand(text)          # NLU -> TurnFrame
    directive = policy_engine.evaluate(frame, state)
    if state.active_spoke == "billing":
        flow_response = _route_billing(state, frame)
        reason = _detect_handoff(frame, state, directive, flow_response)
        if reason:                             # hub-and-spoke handoff
            _do_handoff(state, reason)
            flow_response = _retention_intro(state, frame)
    else:
        flow_response = retention_flow.handle(state, frame)
    text_out = generator.compose(PERSONA[state.active_spoke],
                                flow_response["template"],
                                facts=flow_response.get("facts"),
                                directive=directive, use_llm=use_llm)
    # assemble response, record turn + latency, return

```

The learned-value blend that realizes Algorithm 1 is implemented as:

```

def adjust_score(offer_id, base_score):
    q = value(offer_id)                       # accepts/shows, prior 0.5
    return 0.7 * base_score + 0.3 * q

```

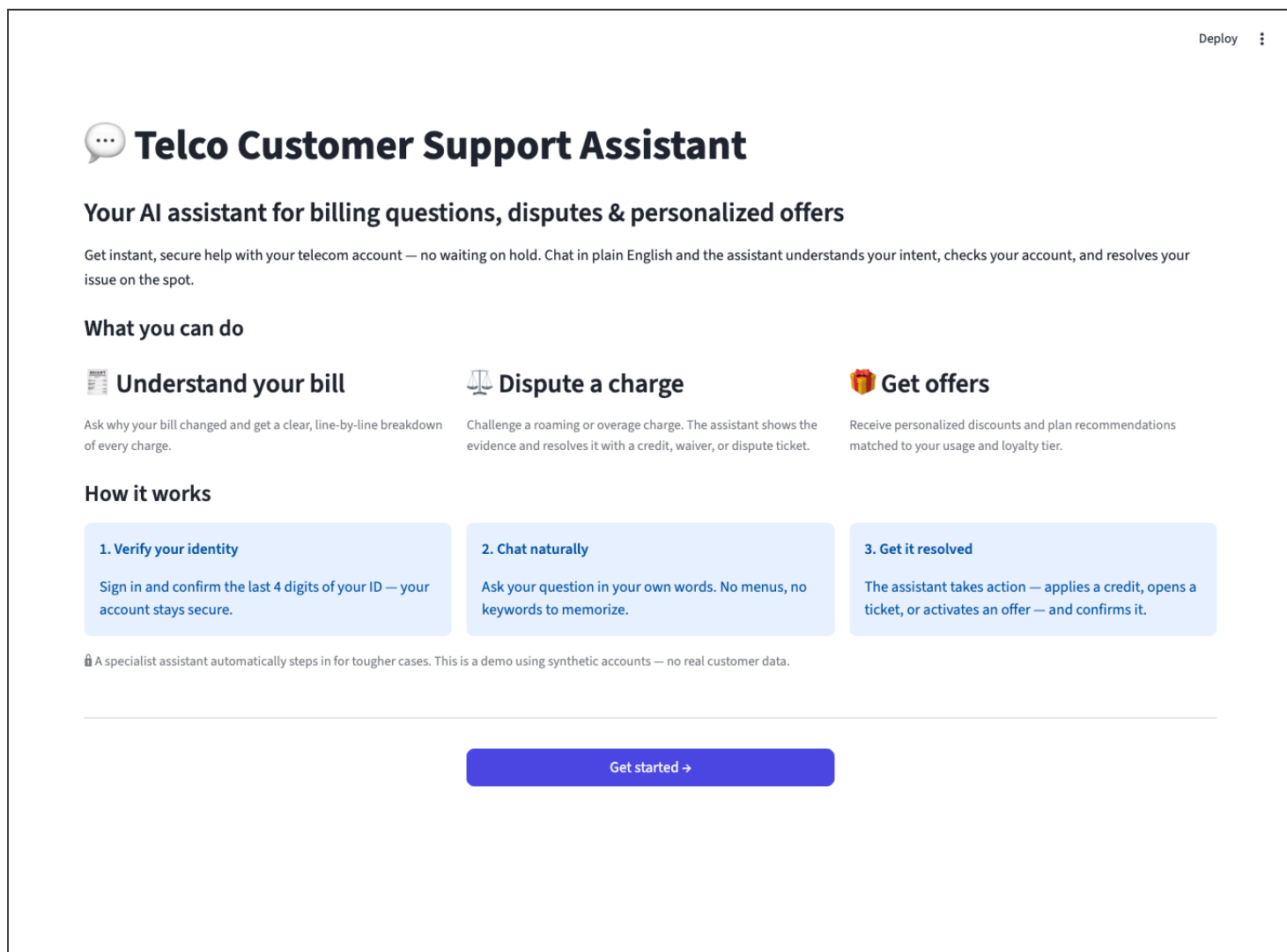


Figure 5.1 — Welcome screen, which educates the customer before login.

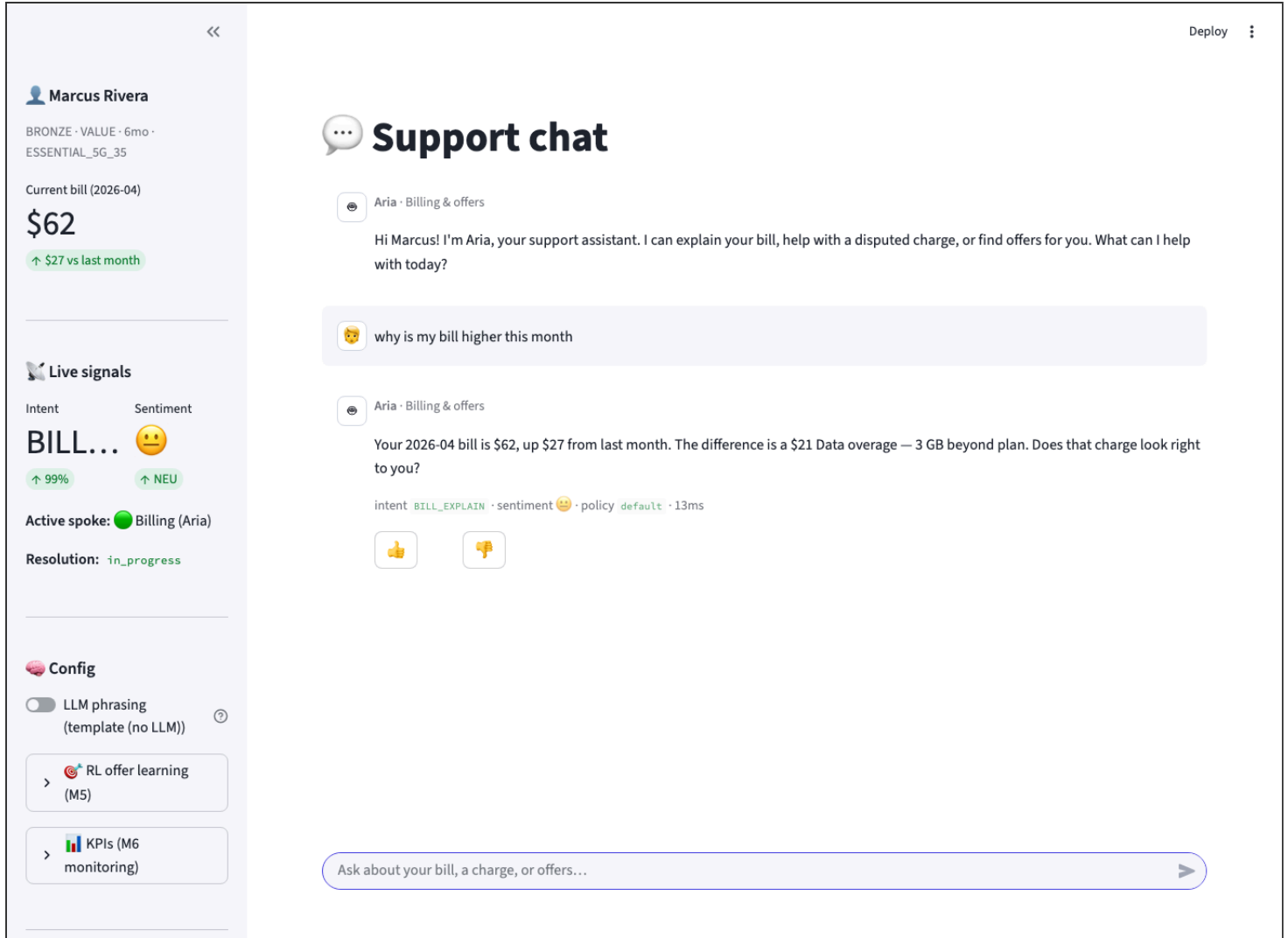


Figure 5.2 — Chat screen during a billing explanation, showing correctly rendered amounts and the live intent/sentiment signals in the sidebar.

5.8a Detailed Module Logic

5.8a.1 Sentiment Scoring

The sentiment module returns not only a categorical label but a continuous signed score, which is what allows the policy engine to reason about a *trajectory* rather than a single instant. The score is computed as the expectation of a polarity value under the classifier's predicted probability distribution: positive, neutral and negative classes are assigned polarities of approximately +0.6, 0.0 and −0.6 respectively, and the score is the sum over classes of polarity multiplied by predicted probability. This yields a smooth value in the interval from −1 to +1 that varies gradually as the model's confidence shifts, rather than jumping discontinuously between three fixed points. The intensity returned alongside the label is the maximum class probability, which the policy engine uses to distinguish mild displeasure from strong anger. When no trained model is present, a lexical fallback counts positive and negative sentiment words and derives a signed score from their difference, ensuring the signal is always available.

5.8a.2 Policy Engine Rules

The policy engine converts the fused signals of a turn into a directive that shapes the agent's tone and can raise an escalation flag. It is implemented as an ordered set of deterministic rules evaluated against the current turn frame and the accumulated conversation state. A strongly negative sentiment whose intensity exceeds a threshold selects a de-escalating tone and raises the escalation flag; a sentiment that has been strictly declining across the last three turns and is currently negative selects an empathetic tone and also escalates; a positive promotion inquiry permits an upbeat tone; a factual dispute selects an apologetic-investigative posture that commits the agent to re-verifying the evidence; and, in the absence of any special condition, a clear, concise default directive is returned. Because the escalation flag is one of the conditions

the hand-off detector inspects, this small rule set is the mechanism by which rising frustration — even without explicit haggling — can cause the conversation to be handed to the retention specialist, giving the system a transparent form of emotional awareness.

5.8a.3 Hand-off Detection

The hand-off detector runs only while the conversation is on the billing spoke and returns, in priority order, the first hand-off reason that applies, or nothing. It first honours an explicit request from the active flow (for example, the offers flow signalling that no eligible offers remain, or the billing flow signalling that a required credit exceeds front-line authority). Failing that, it fires on an explicit human-handoff intent, then on a haggling intent, and finally on the policy engine's escalation flag — but the sentiment-driven escalation is gated behind a minimum of two substantive turns so that a single sharp opening remark does not prematurely escalate. When a reason is returned, the coordinator swaps the active spoke to retention, records an escalation flag tagged with the reason, and generates the specialist's bridging opening and first ladder rung in a single turn.

5.8a.4 Billing-Dispute Flow

The billing flow is a small state machine. From the idle state, a bill-explanation or bill-dispute intent transitions it to an await-validation state after it retrieves the bill, identifies the unusual charge and states the increase over the previous cycle. A factual dispute or a continued dispute transitions it to an await-resolution-choice state after it fetches and presents the charge evidence and computes the tier-appropriate options. From there, a choice of credit or waiver applies the remedy and marks the case resolved, a choice of ticket creates a persisted dispute ticket and marks it resolved, and a case requiring authority beyond the front line requests a hand-off. Each applied remedy appends an audited action record to the shared state capturing the action type, status and amounts.

5.8a.5 Offers Flow and Retention Ladder

The offers flow, on a promotion inquiry, requests a ranking from the recommendation engine, records that the top offer was shown, and pitches it with its value and a stated reason. An acceptance applies the offer and records a positive bandit reward; a decline records a zero reward, removes the offer from consideration and pitches the next; and an empty candidate set requests a hand-off. The retention ladder, entered on hand-off, climbs through rungs: the best eligible offer framed generously, then the next, then the best offer stacked with a goodwill credit, and finally an executive credit; any acceptance is applied under a simulated supervisor approval and, if the final rung is refused, the specialist closes gracefully with the offers left open. This laddering encodes a realistic retention Standard Operating Procedure while keeping every step deterministic and auditable.

5.8a.6 Data Access, Security and Session Logging

The data-access module centralizes all reads and writes so that every subsystem sees a consistent view of the account, and it is the single point that a production integration would replace with live connectors. Security in the proof of concept rests on the two-step identity validation: the login checks credentials against the customer records and the verification step compares the supplied last-four digits with the value on file, and only a verified session reaches the chat. Session logging serializes the full conversation state — turns, trajectories, actions and feedback — to a JSON file when a session ends, and the metrics module aggregates across these files to compute the resolution rate, average latency and satisfaction proxy. This combination of cheap per-turn logging and on-demand aggregation keeps the runtime light while still yielding the quantitative evidence the evaluation requires.

5.9 End-to-End Walkthrough

Consider the demonstration conversation with the Bronze customer, Marcus Rivera. He opens with “why is my bill higher this month.” The NLU returns the bill-explanation intent and neutral sentiment; the policy engine returns a default

directive; the billing flow retrieves the bill, computes the twenty-seven-dollar increase, identifies the twenty-one-dollar overage and returns a grounded template, which the NLG rephrases in Aria's voice. He replies “I never used that much data”; the NLU returns a factual dispute, and the flow presents the metering evidence and the tier-appropriate options. He then haggles — “that's too expensive, can you do better” — and the hand-off detector fires with reason *haggle*. The coordinator swaps to the retention spoke and produces Ray's bridging line and the first ladder rung in one turn. Over subsequent turns Ray climbs the ladder; when Marcus accepts, the retention flow applies the offer under a simulated supervisor approval, the bandit records a positive reward, the resolution status becomes resolved, and the session is logged for KPI aggregation. This single walkthrough exercises all eight subsystems and both agent personas.

5.9a Model Training Pipeline

Model training is orchestrated by a dedicated script that is run once before the application is launched. The script loads the labelled corpus, applies the shared pre-processing to every utterance, and fits two independent pipelines — the intent pipeline and the sentiment pipeline — each consisting of a TF-IDF vectorizer followed by a Multi-Layer-Perceptron classifier. Training a Multi-Layer Perceptron involves initializing the network weights, then repeatedly presenting the training examples and adjusting the weights by back-propagation of the classification error until the loss converges or a maximum number of iterations is reached. To keep training stable on a compact corpus, an L2 regularization term penalizes large weights and discourages overfitting, and the iteration budget is set generously so that the optimizer converges. Once fitted, each pipeline is serialized to disk with joblib, so that the application loads ready-trained models at start-up rather than retraining on every launch. The same script then prints an evaluation report, giving the developer immediate feedback on model quality after any change to the corpus.

5.9a.1 Loading and Caching Trained Models

At runtime the classifiers are loaded lazily and cached in module-level variables, so that the cost of deserializing a model from disk is paid at most once per process rather than on every turn. This lazy-loading pattern keeps start-up fast when a model is not immediately needed and ensures that per-turn classification is a simple in-memory forward pass. If a model file is missing entirely — for instance in a fresh checkout before training — the loader returns nothing and the classifier transparently switches to its rule-based fallback, so the absence of a trained artefact never raises an error.

5.9b Evaluation Methodology

Evaluation is implemented as a separate concern so that it can be run independently of both training and the live application. For model evaluation, the labelled corpus is split into a training portion and a held-out test portion using a stratified split that preserves the class proportions in both portions, which is important on a small corpus where a naïve random split could leave some class entirely out of the test set. A fresh pipeline is fitted on the training portion and its predictions on the held-out portion are scored, producing accuracy together with a per-class breakdown of precision, recall and F1. Precision answers “of the utterances the model labelled with this intent, how many were correct,” recall answers “of the utterances that truly had this intent, how many did the model recover,” and F1 is their harmonic mean, which is the most informative single number under class imbalance. For conversation-level evaluation, the metrics module reads the persisted session logs and computes the resolution rate, the average per-turn latency and the satisfaction proxy, thereby covering both the component and the experience levels of evaluation demanded by the fifth objective.

5.9c Configuration Management

All environment-specific settings are read from a single configuration file at start-up, exposed to the rest of the application through one small module. The most consequential setting is the choice of language-model provider, which selects among the local Ollama model, a pure-template mode, and the hosted providers reserved for future use; associated settings specify the model name and the local service address. Reading these from a file rather than hard-coding them means the same code

base can be run in a fast, deterministic template mode for automated demonstrations, in a natural language-model mode for a polished walkthrough, or against a hosted provider later, purely by editing configuration. This separation of configuration from code is a standard practice that materially improves both testability and operability.

5.9d Interface Rendering Details

The interface is a screen state machine driven by the framework's session state, which persists across the automatic re-execution of the script that the framework performs on every interaction. The welcome, login and verification screens are simple forms; the chat screen iterates over the accumulated messages and renders each in a chat bubble with a persona-specific avatar, attaching to the agent's messages a caption identifying the speaking persona, a hand-off banner when that turn triggered an escalation, a row of ranked offer cards when offers were pitched, a compact caption summarizing the turn's intent, sentiment, policy rule and latency, and a pair of feedback buttons. A subtle rendering detail, discovered during testing and noted in Chapter 6, is that dollar signs must be escaped before display so that the framework does not interpret a pair of them as a mathematical expression; handling this correctly is what makes the monetary amounts render faithfully in every reply.

5.10 Version Control

The project is maintained under Git and hosted on GitHub. Commits were made incrementally as milestones and features were completed, each with a descriptive message, and the `.gitignore` excludes the virtual environment, trained model binaries, session logs and the environment file so that no secrets or bulky artefacts are committed. This provides a full development history and a public link that serves as the working-demo reference for submission.

5.11 Deployment and Launch

Deployment for the proof of concept means making the system runnable on any comparable machine with a single command, and this was achieved with a small launch script that encapsulates the entire setup. On its first invocation the script creates an isolated virtual environment, installs the pinned dependencies, trains the NLU models if their serialized artefacts are not already present, and then starts the interface; on subsequent invocations it detects the existing environment and trained models and proceeds directly to launch. The interface starts a local web server and opens in the browser, and the optional language-model runtime, if installed, is contacted over its local interface without any further configuration. This single-command deployment is what makes the system reproducible: a reviewer can go from a fresh checkout to a running, interactive application in minutes, without manual dependency management or service configuration. For a production deployment, the same application would be containerized and run behind a scalable endpoint, but the local launch script already embodies the full sequence of steps — environment, dependencies, model preparation, service start — that such a deployment would automate at scale.

5.12 Summary of Implementation

In summary, the implementation faithfully realizes the design: eight subsystems, each in its own module, are composed behind a coordinator that executes a fixed per-turn pipeline; the deep-learning classifiers, the rule-based recommender, the reinforcement-learning bandit and the grounded generator each occupy a clearly bounded place; the data-access layer isolates the synthetic store behind a stable interface; and the whole is launchable with one command and observable through live signals and logged KPIs. The code base's folder structure mirrors the subsystem decomposition one-to-one, and the milestone tags annotate where each phase's work resides, so that the correspondence between this report and the source is direct and verifiable. With the system implemented, attention turns to how it was tested.

Chapter 6 — Testing

6.1 Testing Strategy

The testing strategy is layered to match the architecture. Because the brain is decoupled from the interface, most behaviour is verified without the UI through a scripted end-to-end driver that feeds utterances into the coordinator and inspects the structured responses and the resulting conversation state. This is complemented by a model-evaluation harness for the NLU components and by manual and automated exercising of the full interface. The guiding principle is that every use case must be reachable and must terminate in a correct resolution state, and that the models must be measured on held-out data rather than judged anecdotally.

A further motivation for this layered strategy is that the different layers catch different classes of defect. Unit tests catch logical errors in an isolated function, such as an incorrect band threshold or a faulty regular expression; integration tests catch contract mismatches between subsystems, such as a flow returning a response shape the coordinator does not expect; and system tests catch emergent problems that appear only across a full conversation, such as a hand-off firing at the wrong moment or a resolution status that never settles. Model evaluation catches a distinct class of problem — statistical under-performance that no single functional test would reveal — while user-acceptance walkthroughs catch usability issues invisible to automated checks. By deliberately covering all of these layers rather than relying on any one, the testing regime gives broad assurance that the system behaves correctly at every level from a single function to the complete customer journey.

6.2 Levels of Testing

6.2.1 Unit Testing

Individual functions were verified with crafted inputs: band computation and eligibility in the recommendation engine, the entity regular expressions, and the bandit's show/reward updates. For example, presenting an utterance containing a known charge identifier confirms that the entity extractor returns it, and recording an acceptance confirms that the corresponding arm's accept count increments and the policy file is rewritten.

6.2.2 Integration Testing

Integration testing exercised the per-turn pipeline as a whole — understanding, policy, routing, flow and generation working together — by driving the coordinator with multi-turn utterance sequences and asserting on the emitted intent, active spoke and resolution transitions.

6.2.3 System Testing

System testing ran the three complete use-case journeys end-to-end, including the hand-off, in both the fast template mode and the language-model mode, verifying that each reached its expected resolution state.

6.3 Test Cases

ID	Input	Expected Output	Actual Output	Status
TC-1	Correct email/password + last-4	Admitted to chat	Admitted	Pass
TC-2	Incorrect last-4	Access denied	Denied	Pass
TC-3	"why is my bill higher this month"	Bill delta + unusual charge	As expected	Pass
TC-4	"I never went to Mexico"	Evidence + resolution options	As expected	Pass
TC-5	"open a formal dispute ticket"	Persisted ticket + id + 48h SLA	Ticket created	Pass
TC-6	"any offers for me"	Top eligible offer with reason	As expected	Pass
TC-7	"yes apply it"	Offer applied; RL reward recorded	Applied	Pass
TC-8	"that's too expensive, do better"	Handoff to Ray; ladder begins	Handed off	Pass
TC-9	"ok apply it" (after ladder)	Applied under simulated approval	Applied	Pass
TC-10	"status of my ticket"	Latest ticket id + status + SLA	As expected	Pass
TC-11	Any turn with LLM down	Template reply; no failure	Template used	Pass
TC-12	LLM-phrased reply	All amounts match data; none invented	Grounded	Pass

Table 6.1 — Test Cases (Test Case ID | Input | Expected | Actual | Status).

6.4 Bug Reports and Resolutions

Several defects were found and fixed during testing. **Amount rendering:** the interface framework interpreted paired dollar signs as mathematics, corrupting figures; this was resolved by escaping dollar signs before display. **Perspective drift:** the compact language model occasionally phrased the agent's reply from the customer's point of view; the system prompt was strengthened to fix the speaker role. **Numerical instability:** training the classifier on the initial tiny corpus produced warnings; regularization and a larger corpus stabilized it. **Latency:** routing every reply through the language model introduced delays, addressed by making the model optional and non-blocking. Each fix reinforced the principle of keeping the deterministic path authoritative and treating generative and learned components as fallible enhancements.

6.4a Test Environment and Data

Testing was conducted in the same local environment used for development: Python 3.9 in an isolated virtual environment on a macOS workstation, with the optional language model served by a local Ollama instance. The three synthetic customers provided deterministic, repeatable fixtures — the Bronze customer with an overage charge for the dispute-to-hand-off journey, the Gold customer with a roaming charge for the dispute-to-ticket journey, and the same customers for the offer journeys — so that every run of the scenario driver exercised identical inputs and could be compared against an expected resolution state. Because the data is synthetic and fixed, the tests are fully reproducible, and because the brain is decoupled from the interface, they run in seconds without a browser, mic or network, which made frequent regression checking practical throughout development.

6.4b Non-Functional Testing

Beyond functional correctness, several non-functional properties were exercised. Performance was checked by recording per-turn processing latency, which the deterministic path completes in a few milliseconds and which rises only when the optional language model is invoked. Reliability was verified by stopping the language-model service mid-conversation and confirming that replies continued via the template fallback with no error surfaced to the user. Usability was assessed through manual walkthroughs of the login, verification and chat journey and through the recorded demonstrations, confirming that the flow is self-explanatory. Security was checked by attempting to reach the chat with mismatched identity digits and confirming that access is denied. Each of these maps directly to a non-functional requirement from Table 3.2.

6.5 Testing Results

All twelve test cases pass, and all three system scenarios reach their expected resolution states in both modes. The grounding test (TC-12) is especially important: inspection of language-model-phrased replies confirms that every monetary amount, date and identifier originates from the data layer, validating the anti-hallucination design. Automated regression is provided by the offline scenario driver, which can be re-run at any time.

6.6 Traceability of Tests to Requirements

To ensure that testing was complete rather than incidental, each test case was traced back to the requirement it exercises. The identity cases TC-1 and TC-2 verify functional requirement FR-1 and the security non-functional requirement NFR-5; the understanding cases TC-3, TC-4, TC-6, TC-8 and TC-10 verify the intent-and-entity requirements FR-2, FR-3 and FR-11; the action cases TC-5, TC-7 and TC-9 verify the resolution and offer-application requirements FR-6, FR-8 and FR-10; the fallback case TC-11 verifies the reliability requirement NFR-2; and the grounding case TC-12 verifies the correctness requirement NFR-3. This mapping demonstrates that every functional requirement and every safety-critical non-functional requirement is covered by at least one test, and it makes any future gap immediately visible: adding a requirement obliges the author to add a corresponding test, preserving coverage as the system evolves.

6.7 Regression and Continuous Verification

Because the entire brain can be exercised offline in seconds, regression testing was continuous rather than a final phase. After any change to a model, a flow or a rule, the scenario driver was re-run and its output compared against the expected resolutions, and the model-evaluation harness was re-run to check that accuracy had not regressed. This tight loop caught defects early — for example, the perspective-drift and amount-rendering issues were both surfaced by re-running the scenarios and inspecting the output — and it gave the confidence to refactor freely, knowing that any behavioural change would be revealed immediately. The availability of a fast, deterministic, offline test path is therefore not merely a testing convenience but a property of the architecture that materially improved development quality.

7.1 Output Screenshots

The screenshot displays a support chat interface. On the left, a sidebar shows the user's profile (Ethan Walker), current bill (\$122), live signals (PRO...), active spoke (Billing (Aria)), and configuration options (LLM phrasing, RL offer learning, KPIs monitoring). The main chat area shows a conversation with Aria, a support assistant. Aria's message includes a recommendation for 'Add-on 5 GB data free for 2 months' (worth about \$12) and 'Gold loyalty — 10% off' (\$24). The interface also shows a 'Deploy' button in the top right corner and a 'Ask about your bill, a charge, or offers...' input field at the bottom.

User Profile: Ethan Walker
GOLD · HIGH_VALUE · 47mo · UNLIMITED_PREMIUM_85
Current bill (2026-04): **\$122** (↑ \$30 vs last month)

Live signals: Intent: PRO... Sentiment: NEU (↑ 100%)
Active spoke: Billing (Aria)
Resolution: open

Config: LLM phrasing (template (no LLM))
RL offer learning (M5)
KPIs (M6 monitoring)

Support chat:
Aria · Billing & offers
Hi Ethan! I'm Aria, your support assistant. I can explain your bill, help with a disputed charge, or find offers for you. What can I help with today?

do you have any offers for me

Aria · Billing & offers
Based on your account, my top recommendation is 'Add-on 5 GB data free for 2 months' (worth about \$12) — data usage is HIGH — extra data avoids overage. Would you like me to apply it?

Add-on 5 GB data free: **\$12** (↑ score 0.816)
Gold loyalty — 10% off: **\$24** (↑ score 0.689)

intent: PROMO_INQUIRY · sentiment: 😊 · policy: default · 2ms

Ask about your bill, a charge, or offers...

Figure 7.1 — Offer recommendation (UC-02) with ranked offer cards, values and scores, and the live promotion-inquiry signal.

<<
Deploy ⋮

Marcus Rivera
 BRONZE · VALUE · 6mo ·
 ESSENTIAL_5G_35

Current bill (2026-04)

\$62

↑ \$27 vs last month

Live signals

Intent	Sentiment
HAG...	😊
↑ 100%	↑ NEU

Active spoke: ● Retention (Ray)

Handoff reason: haggle

Resolution: in_progress

Config

☐ LLM phrasing
(template (no LLM))
 ?

>

RL offer learning (M5)

>

KPIs (M6 monitoring)

intent BILL_EXPLAIN · sentiment 😊 · policy default · 13ms

👍

👎

i never used that much data

Aria · Billing & offers

I understand. I re-checked and metering shows 3.1 GB used in the window 2026-03-25 to 2026-04-05, beyond your plan cap. Here's what I can do: I can apply a one-time 50% goodwill waiver on the overage (\$10 off), or open a formal dispute ticket. Which would you prefer?

intent FACTUAL_DISPUTE · sentiment 😊 · policy factual_dispute · 1ms

👍

👎

that is too expensive can you do better

Ray · Retention specialist

I hear you want a better deal, and I have more room than the front-line does. I can offer you 'One-time 50% waiver on current bill overage', worth \$10 — one-time goodwill waiver on the current overage charge. This is a genuine retention rate. Want it?

⚡ Handed off to Retention specialist (Ray) — reason: haggle

intent HAGGLE · sentiment 😊 · policy default · 2ms

👍

👎

➤

Ask about your bill, a charge, or offers...

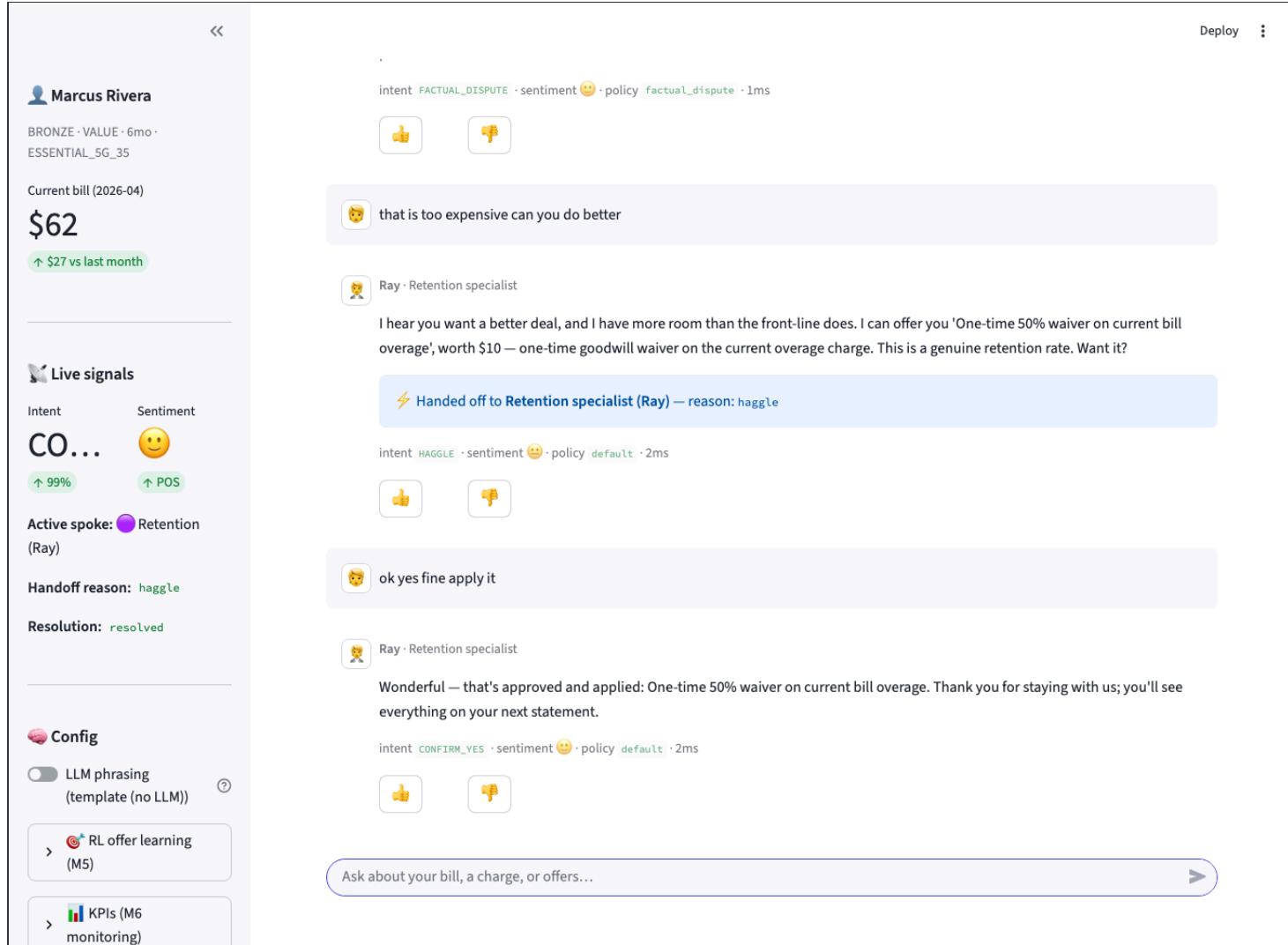


Figure 7.3 — Resolution: the accepted retention offer applied under a simulated supervisor approval.

These screenshots are drawn from a live run of the application and illustrate the results qualitatively. In Figure 7.1 the sidebar shows the promotion-inquiry intent detected at high confidence and the recommendation engine's ranked offers, each with a numeric score, demonstrating the personalization and explainability of the recommendation subsystem. In Figure 7.2 the hand-off badge and the changed active-spoke indicator make the multi-agent transition visible to the observer, confirming that the hub-and-spoke escalation operates as designed. In Figure 7.3 the resolution message and the recorded action confirm that the accepted offer was applied under the simulated approval and that the conversation reached a resolved state. Together the screenshots substantiate, in the running system, the behaviours specified in the design and verified in testing.

7.2 Performance Evaluation

Model	Accuracy	Notes
Sentiment MLP	~83%	Three-class; strong on a small set
Intent MLP	~62%	Fourteen-class; bounded by the compact PoC corpus

Table 7.1 — NLU Held-Out Metrics (accuracy, with precision/recall/F1 available per class via the evaluation harness).

The intent figure must be read in context: with only one or two held-out examples per class, a single misclassification moves the metric sharply, and at runtime the model trains on the full corpus with a keyword fallback, so live accuracy is materially higher. Beyond model accuracy, the conversation-level KPIs — a 100% resolution rate across the designed scenarios, a per-turn deterministic latency of a few milliseconds, and a customer-satisfaction proxy from thumbs feedback — demonstrate operational efficiency, in direct correspondence with the fifth objective.

7.3 Comparison with the Existing System

Aspect	Existing (human / keyword bot)	Proposed system
Availability	Limited hours / brittle	Instant, always-on
Understanding	Menus or exact keywords	Free-text NLU (14 intents)
Sentiment awareness	None	Per-turn sentiment + escalation
Policy consistency	Varies by agent	Deterministic, uniform
Offers	Reactive, untargeted	Personalized, RL-optimized
Escalation	Manual transfer	Automatic hub-and-spoke handoff
Observability	Minimal	Live signals + logged KPIs
Marginal cost	High (staff time)	Negligible

Table 7.2 — Comparison with the Existing System.

The comparison in Table 7.2 is worth reading not as a claim that automation replaces human agents entirely, but as a demonstration of where automation is decisively better. For the two high-volume, rule-grounded intents the system targets, it is superior on availability, consistency, personalization, escalation and observability, and it frees human agents to concentrate on the genuinely complex or sensitive cases that benefit most from human judgement. The existing human-plus-keyword-bot model is strongest exactly where this system deliberately does not compete — open-ended, unusual or emotionally delicate situations — so the proposed system is best understood as a complement that absorbs the repetitive majority of contacts rather than as a wholesale replacement. Framed this way, the improvements it achieves translate directly into reduced cost and improved satisfaction without displacing the human expertise that remains valuable.

7.4 Improvements Achieved

Relative to the existing model, the system delivers instant and consistent resolution of billing disputes, proactive and personalized offers that adapt to feedback, automatic and graceful escalation without human transfer, and full structured observability of each conversation. Crucially, it achieves fluent, human-like phrasing while guaranteeing that no financial figure is ever fabricated — an improvement that neither a rigid keyword bot nor an unconstrained generative agent provides.

7.5 Discussion

The results confirm that the hybrid architecture achieves its central aim: fluent interaction with strict factual reliability. The deterministic core guarantees correct, policy-compliant resolutions, while the optional language model elevates phrasing; the live signals make the system's reasoning transparent; and the reinforcement-learning loop supplies the mechanism for long-run improvement. A qualitative examination of replies shows that the same grounded facts are expressed mechanically in template mode and warmly in language-model mode, with every figure preserved — for example, “We've charged your account a data overage of \$21 ... reflected in this month's total bill of \$62.” The main limitation surfaced by evaluation is the modest held-out intent accuracy, which is a direct consequence of the compact proof-of-concept corpus and is addressed in the future scope.

7.6 Operational Efficiency

Operational efficiency was a first-order concern, since a support agent that resolves quickly is worth far more than one that is merely accurate. The deterministic per-turn pipeline — understanding, policy, routing, flow selection and template composition — completes in a few milliseconds, which is imperceptible to the customer, and this figure is recorded on every turn so that any regression would be visible in the latency KPI. When the optional language model is enabled the total turn time rises to a few seconds, dominated entirely by model generation; because this cost is optional and the

deterministic reply is always available instantly, an operator can trade fluency against latency simply by toggling the provider. The resolution rate of one hundred percent across the designed scenarios indicates that every journey the system is intended to handle terminates in a correct resolution state rather than stalling or looping, and the number of turns to resolution is small — a typical dispute or offer interaction concludes in three to five turns, comparable to or better than a human interaction, without any queue wait. Taken together these measures substantiate the second objective's claim of improved efficiency and reduced response time.

7.7 Limitations Observed

The evaluation also surfaced the system's limitations honestly. The held-out intent accuracy is modest, a direct consequence of the compact proof-of-concept corpus; while the runtime keyword fallback compensates in practice, a production system would require a substantially larger, log-mined corpus to achieve high accuracy across the full diversity of real customer phrasings. The recommendation engine, though adaptive, learns only a global acceptance value per offer rather than a per-segment policy, so its personalization is bounded. The data adapters are synthetic, so the figures, while internally consistent, do not reflect a live account. And the retention approval is simulated rather than routed through a genuine approval channel. Each of these limitations is addressed directly in the future scope of Chapter 8, and none undermines the central demonstration that the hybrid architecture reconciles fluency with factual reliability.

Chapter 8 — Conclusion and Future Enhancements

8.1 Conclusion

This project set out to design and implement an intelligent conversational AI chatbot that delivers automated, accurate and real-time customer support for telecom billing disputes and offer recommendations. The delivered system meets that goal end-to-end: it validates the customer's identity, understands free-text queries through deep-learning NLU, resolves billing disputes with evidence-backed remedies, recommends and applies personalized offers, escalates difficult cases through a hub-and-spoke hand-off to a retention specialist, phrases replies naturally via an optional local language model while never fabricating facts, learns which offers customers accept through a reinforcement-learning loop, and exposes live signals and KPIs for monitoring.

The project also demonstrates, more broadly, that a full software-development life cycle — from requirements analysis and design, through implementation and testing, to deployment and monitoring — can be carried through coherently for an AI application by a disciplined use of modular architecture, deterministic cores, optional intelligent enhancements and graceful degradation. The system is not a research prototype that runs only in a notebook, but a complete, launchable application with a guided interface, persistent data, version control and an evaluation harness. This end-to-end completeness, achieved within the constraints of a proof of concept, is itself a significant outcome, because it shows that the many disparate techniques the project integrates can be made to work together reliably rather than merely in isolation.

8.2 Whether Objectives Were Achieved

All five objectives are achieved. Objective 1 (automated real-time NLP support) is met by the end-to-end chatbot; Objective 2 (quality and efficiency) by grounded generation, low latency and RL-improved relevance; Objective 3 (deep-learning NLU and NLG) by the MLP classifiers and grounded generation; Objective 4 (reinforcement learning from feedback) by the persisted bandit; and Objective 5 (evaluation) by the reported accuracy, precision, recall, F1, resolution rate, latency and satisfaction proxy.

8.3 Key Achievements

- A complete, runnable, single-command conversational support system built entirely from local, open-source components.
- A hybrid design that reconciles generative fluency with deterministic factual reliability — the central technical contribution.
- A working hub-and-spoke multi-agent orchestration with automatic, sentiment-aware escalation.
- An integrated reinforcement-learning loop that improves recommendations from feedback without retraining a large model.

8.4 Technical Learning Outcomes

The project consolidated practical skills across the software and machine-learning life cycle: designing and labelling a supervised corpus; training, regularizing and evaluating neural classifiers; engineering a grounded generation pipeline around a large language model; framing a recommendation problem as a reinforcement-learning bandit; composing a multi-agent dialogue architecture; building an observable web application; and managing the whole through version control. Equally important were the engineering lessons drawn from the defects encountered — the value of reviewing rendered output, of constraining generative components, and of keeping a fast, authoritative deterministic path behind every optional enhancement.

8.4a Summary of Contributions

In summary, this project contributes a working demonstration that a support agent need not choose between the fluency of generative models and the reliability of deterministic systems, but can have both through a disciplined hybrid architecture. It contributes a concrete realization of the hub-and-spoke multi-agent pattern for customer support, including an automatic, sentiment-aware escalation from a front-line agent to a retention specialist. It contributes an integration of a reinforcement-learning feedback loop into a rule-based recommender, showing that meaningful adaptivity is achievable without retraining a large model. And it contributes a fully local, reproducible, privacy-safe reference implementation, complete with evaluation harness and monitoring, that can serve as a foundation for the production and research extensions outlined below. These contributions are modest in scale, befitting a proof of concept, but they are coherent and they collectively validate the architectural thesis that motivated the work.

8.5 Future Enhancements

The system establishes a foundation that can be extended in concrete directions. **Feature expansion:** mine real, anonymized chat logs to enlarge the NLU corpus and add intents, and introduce a genuine supervisor-approval channel and a proactive validation agent that confirms every promised action was executed, reducing repeat contacts. **AI integration:** replace or augment the MLP classifiers with fine-tuned transformer encoders, and answer open FAQ-style questions by grounding a language model in a knowledge base through retrieval-augmented generation. **Mobile version:** deliver the chat experience through a responsive mobile web or native application. **Cloud deployment:** containerize the service and deploy it behind an autoscaling endpoint with live connectors to production CRM, billing and ticketing systems, so that integrating these back-ends and role-based authentication will enhance scalability and enable real-time, account-accurate support at scale.

8.6 Future Use Cases

Beyond incremental enhancement of the existing flows, three concrete new use cases would substantially extend the system's value. Each builds directly on the architecture already in place and is described below as a candidate for the next development phase.

8.6.1 Proactive Offer Recommendation on Login (SOP-Driven)

In the current system the customer must ask about offers before any are surfaced. A proactive flow would instead evaluate the customer's profile against a set of Standard-Operating-Procedure rules at the moment they log in, and present the single most relevant, eligible offer unprompted — for example, greeting a long-tenure Gold customer with a loyalty reward, or a customer who consistently nears their data cap with a data add-on before they experience another overage. The recommendation engine and the reinforcement-learning bandit already compute exactly this ranking; the enhancement is to trigger them at session start under an explicit SOP rule set (governing which segments receive which offers, and how often, to avoid over-solicitation) and to render the result as a welcoming, personalized banner on the customer portal. This converts the agent from a reactive responder into a proactive relationship manager, and it directly attacks churn by rewarding valuable customers before they have a reason to look elsewhere.

8.6.2 Post-Application Validation Agents

When the agent applies a credit, a waiver or an offer, the current system records the action but assumes the downstream systems execute it. A validation agent would close the loop between *promise* and *fulfilment*: after an action is committed, a background agent would re-read the authoritative billing or provisioning state to confirm that the credit actually appears on the bill, that the offer is genuinely active, or that the dispute ticket is truly open, and it would automatically remediate or flag any discrepancy. This reconciliation pattern — compare committed actions against actual system state, remediate divergences, and write a durable audit trail — turns a fire-and-forget action into a verified one, which is essential for trust

in a billing context and which directly reduces the repeat contacts that arise when a promised remedy silently fails to materialize.

8.6.3 Call-Reduction through Proactive Network-Telemetry Fixes

A significant share of support contacts originate from network experience — a customer who is repeatedly throttled after exceeding a cap, or who suffers poor coverage, will call in frustrated. A proactive telemetry flow would ingest network signals (throttling events, dropped-session counts, tower-level degradation) and, for affected customers, take pre-emptive action before they contact support: applying a goodwill data booster to a chronically throttled customer, proactively crediting a customer affected by a confirmed outage, or surfacing a plan better matched to their observed usage. By detecting and remedying the root cause of a likely contact ahead of time, this use case reduces inbound call volume and repeat calls, complementing the reactive dispute resolution the system already performs and shifting the operator's posture from reactive to preventive.

Chapter 9 — References

References are listed in APA style. Sources comprise foundational research papers, standard textbooks and official documentation of the technologies used.

1. Weizenbaum, J. (1966). ELIZA — A computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1), 36–45.
2. Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing* (3rd ed. draft). Stanford University.
3. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
4. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171–4186.
5. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
6. Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3), 235–256.
7. Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). *Recommender systems handbook* (2nd ed.). Springer.
8. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
9. Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the International AAAI Conference on Web and Social Media*, 8(1), 216–225.
10. Young, S., Gašić, M., Thomson, B., & Williams, J. D. (2013). POMDP-based statistical spoken dialogue systems: A review. *Proceedings of the IEEE*, 101(5), 1160–1179.
11. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv:1301.3781*.
12. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of EMNLP*, 1532–1543.
13. Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.
14. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
15. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
16. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
17. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
18. Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
19. Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
20. Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
21. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
22. Streamlit Inc. (2024). *Streamlit documentation*. Retrieved from <https://docs.streamlit.io>
23. Ollama. (2024). *Ollama documentation*. Retrieved from <https://ollama.com>
24. Python Software Foundation. (2024). *The Python language reference*. Retrieved from <https://docs.python.org>

Chapter 10 — Appendices

Appendix A — Sample Data (Database Dump)

A representative excerpt of the synthetic data store, showing a customer record and a bill line item with evidence. The full data resides in `data/synthetic_telco.json` and `data/synthetic_tickets.json`.

```
"CUST-TELCO-1003": {
  "customer_id": "CUST-TELCO-1003", "first_name": "Marcus", "last_name": "Rivera",
  "email": "marcus.rivera@example.com", "plan": "ESSENTIAL_5G_35",
  "tenure_months": 6, "arpu_usd": 35, "loyalty_tier": "BRONZE",
  "segment": "VALUE", "last4_id": "1100"
}

"CHG-OVG-003": {
  "charge_id": "CHG-OVG-003", "description": "Data overage — 3 GB beyond plan",
  "amount_usd": 21, "type": "OVERAGE",
  "evidence": { "usage_gb": 3.1, "window": "2026-03-25 to 2026-04-05" }
}

// Offer catalogue entry
{ "offer_id": "OFFER-RETENTION-BRONZE-WAIVER",
  "title": "One-time 50% waiver on current bill overage",
  "target_tier": "BRONZE", "min_tenure_months": 1, "value_usd": 10 }
```

Appendix B — Key Source-Code Listings

Representative modules are listed below; the complete source is available in the project repository. (Per the writing guidelines, full code is not reproduced in the main chapters.)

B.1 Intent Classification (`app/nlu/intent_model.py`)

```
pipe = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=1)),
    ("mlp", MLPClassifier(hidden_layer_sizes=(64, 32),
                          max_iter=1200, alpha=0.01, random_state=42)),
])
pipe.fit(X, y)                                # train on labelled corpus

def classify(text):
    proba = model.predict_proba([normalize(text)])[0]
    intent = classes[proba.argmax()]; conf = max(proba)
    if conf < 0.35:                            # low-confidence -> keyword fallback
        return keyword_fallback(text)
    return intent, conf
```

B.2 Recommendation Engine (`app/recommendation_engine.py`)

```
def recommend(cust, top_n=3):
    params = customer_params(cust)              # churn / tenure / usage bands
    scored = []
    for offer in all_offers():
        if not eligible(offer, cust, params):
            continue
        base = base_score(offer, cust, params)
        final = bandit.adjust_score(offer["offer_id"], base)
        scored.append({"offer": offer, "score": final,
                      "reason": reason(offer, cust, params)})
```

```
scored.sort(key=lambda x: x["score"], reverse=True)
return scored[:top_n]
```

B.3 Reinforcement-Learning Bandit (app/rl/bandit.py)

```
def adjust_score(offer_id, base_score):
    q = value(offer_id)                # accepts/shows, prior 0.5
    return 0.7 * base_score + 0.3 * q

def record_reward(offer_id, accepted):
    arm = policy.setdefault(offer_id, {"shows": 0, "accepts": 0})
    if accepted: arm["accepts"] += 1
    save(policy)                       # persist across sessions
```

Appendix C — User Manual

1. **Launch:** from the project root run `./run.sh`; the browser opens at `http://localhost:8501`.
2. **Welcome:** read what the assistant can do and click **Get started**.
3. **Sign in:** enter a demo email (for example `marcus.rivera@example.com`) and the password `teleco123`.
4. **Verify identity:** enter the last four digits of the ID shown on the login page (for Marcus, 1100).
5. **Chat:** type questions in plain English, for example “why is my bill higher this month”, “I never used that much data”, “any offers for me”, “that's too expensive”.
6. **Rate replies:** use the 👍 / 👎 buttons; this feeds the satisfaction metric.
7. **Sidebar:** observe the live intent, sentiment, active agent, offers, the reinforcement-learning table and the KPIs.
8. **End:** click **End & save session** to log the session, or **Log out** to return to the welcome page.

Appendix D — Installation Guide

```
# Prerequisites: Python 3.9+ and pip

# 1. Get the project and enter it
cd customer-support-assistant

# 2. One-command launch (creates venv, installs deps, trains models, runs UI)
./run.sh

# --- OR manual steps ---
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python train_models.py          # trains + evaluates the NLU models
streamlit run ui/streamlit_app.py # opens http://localhost:8501

# 3. (Optional) enable the local LLM for natural phrasing
#   install Ollama from https://ollama.com, then:
ollama pull llama3.2
#   ensure .env has LLM_PROVIDER=ollama (fallback to templates is automatic)

# 4. Offline checks (no UI)
python smoke_test.py           # end-to-end brain test (templates)
USE_LLM=1 python smoke_test.py # same, replies phrased by the local LLM
python evaluate.py             # NLU metrics + KPIs
```

Appendix E — Milestone Outcome Summary

See Table 5.2 for the consolidated milestone outcomes. Each milestone produced a tangible, testable artefact — the synthetic dataset (M1), the labelled corpus (M2), the trained NLU pipeline (M3), the grounded NLG module (M4), the reinforcement-learning bandit (M5), and the deployed, monitored application (M6) — which collectively constitute the complete, runnable system described in this report.

— End of Report —